

Project3

Regression

Team 12 - Steffi Dorothy

Task 1 - Introduction

1.1 Background and Significance

CalCOFI is a long-standing oceanographic program that monitors the California Current system, providing vital data for understanding marine ecosystems and climate.

1.2 The Dataset

Rich Repository: CalCOFI's bottle database is a vast collection of oceanographic data, including temperature, salinity, depth, nutrients, and more, gathered over many years.

Extensive Dataset: 864863 records

Diverse Attributes: 74 columns

1.3 Motivation

Our project aims to harness the power of this extensive dataset to develop predictive models for oceanographic parameters, focusing specifically on the relationships between temperature, salinity, and depth. These parameters are fundamental to understanding ocean circulation, climate change impacts, and marine ecosystem health. By leveraging machine learning techniques, we hope to create models that can accurately predict temperature variations based on salinity and depth measurements.

1.4 Potential Impact

A successful solution could revolutionize the way we monitor and predict ocean conditions. Accurate predictive models can enhance our understanding of ocean dynamics, contribute to climate change research, and support marine conservation efforts. For instance, anticipating temperature fluctuations could help manage fisheries, protect endangered species, and mitigate climate change's effects on marine habitats.

1.5 Anticipated Challenges

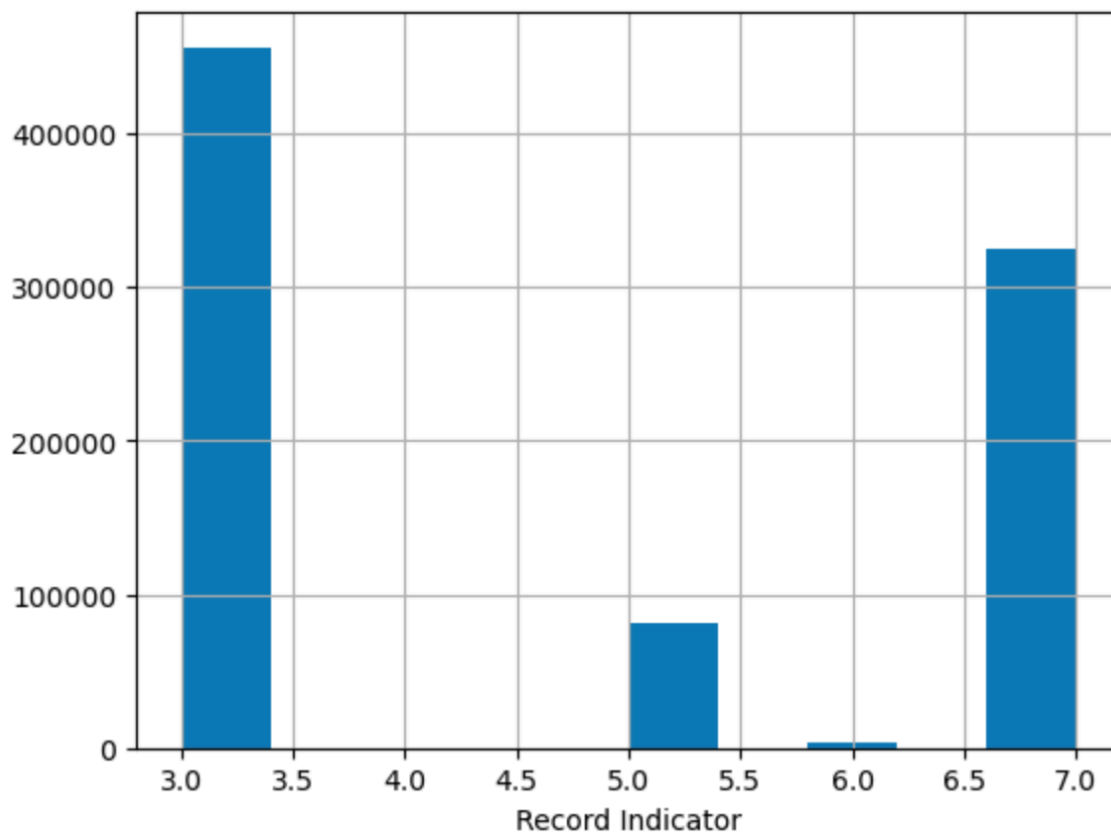
However, this project has its challenges. The complexity and variability of oceanographic data pose significant hurdles in model accuracy and generalizability. Missing values, noisy data, and the high dimensionality of the dataset require robust preprocessing and sophisticated modeling techniques. Additionally, capturing the intricate interactions between multiple parameters demands careful feature selection and model tuning.

Task 2 - Exploratory Data Analysis (EDA)

2.1 Key: Bottle Table

Field Name	Description
Cst_Cnt	Cast Count - All CalCOFI casts ever conducted, consecutively numbered
Btl_Cnt	Bottle Count - All CalCOFI bottles ever sampled, consecutively numbered
Depthm	Bottle depth in meters
T_degC	Water temperature in degrees Celsius
Salnty	Salinity (Practical Salinity Scale 1978)
RecInd	Record Indicator (quality code that applies to the whole bottle, instead of just to a specific parameter)
T_prec	Temperature Precision
S_prec	Salinity Precision

NH3q	Quality Code Of Ammonia Concentration
R_Depth	Reported Depth (from pressure) in meters
R_PRES	Reported Pressure - pressure at which a water sample was collected.



2.2 The histogram shows the distribution of the number of segments (RecInd) for a given dataset.

Distribution Shape:

- The distribution is skewed to the right, meaning a longer tail is on the right side. This indicates that a few individuals have a significantly higher number of segments than the majority.

Mode:

- The mode (most frequent value) appears to be around 3.0. This suggests that the majority of individuals have 3 segments.

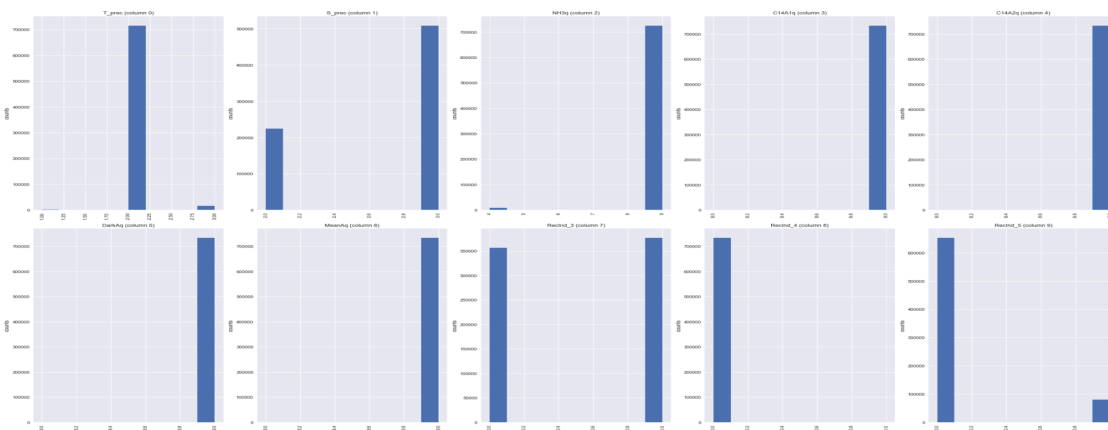
Range:

- The data ranges from approximately 3.0 to 7.0. This indicates that the minimum number of segments is 3, and the maximum number is 7.

Outliers:

- There are a few outliers on the right side of the distribution, representing individuals with a much higher number of segments than the typical values. These outliers might warrant further investigation to understand the reasons for their extreme values.

2.2 Distribution Graphs Of Sampled Columns



Analysis of Histograms:

1. Temperature precision (T_prec) (Column 0):

- **Observation:** The majority of the data points are concentrated around a specific value, with very few outliers.
- **Implication:** This suggests that the temperature precision (T_prec) is mostly consistent across the dataset, with minimal variability.

2. Salinity precision (S_prec) (Column 1):

- **Observation:** Similar to T_prec, the salinity precision (S_prec) shows a high concentration around a central value with very few outliers.
- **Implication:** This indicates that the salinity measurements are also consistent, with little variation.

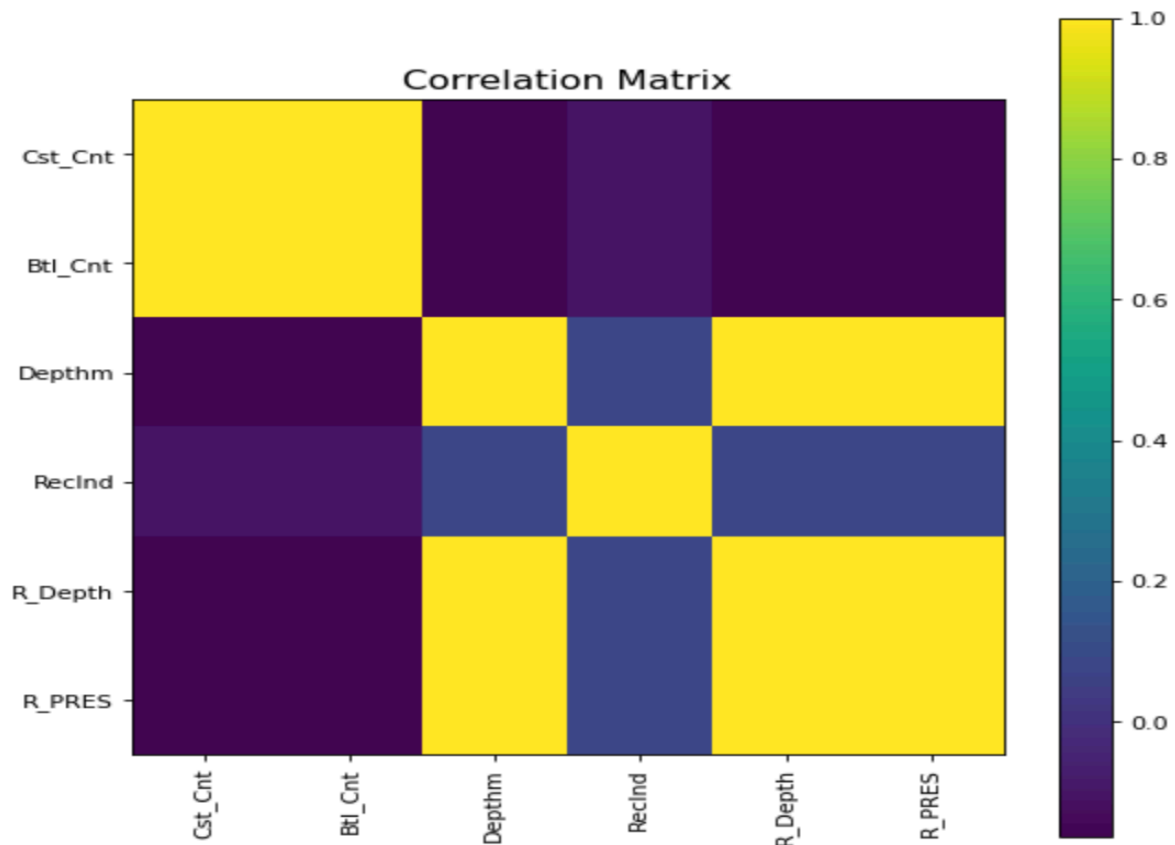
3. Ammonia concentration (NH3q) (Column 2):

- **Observation:** There is a high concentration of values around a central point with almost no variability.
- **Implication:** The ammonia concentration (NH3q) appears to be very stable in the dataset, which could be indicative of a controlled environment or consistent sampling conditions.

High Concentration of Values: Most of the columns show a high concentration of values around a central point with minimal variability. This could imply that the conditions under which the data was collected were highly controlled or that the parameters being measured are inherently stable.

Minimal Variability: The lack of variability across multiple parameters might suggest that the dataset could be from a highly controlled environment or that the parameters themselves do not exhibit much natural variability.

Potential for Redundancy: Columns with almost identical distributions might indicate redundant information, which could be streamlined during the data preprocessing stage.



2.3 The provided correlation matrix visualizes the relationships between different variables:

Cast Count (Cst_Cnt), Bottle Count(Btl_Cnt), Depthm, Record Indicator (Recind), Reported Depth (from pressure) in meters(R_Depth), and R_PRES.

Strong Positive Correlations:

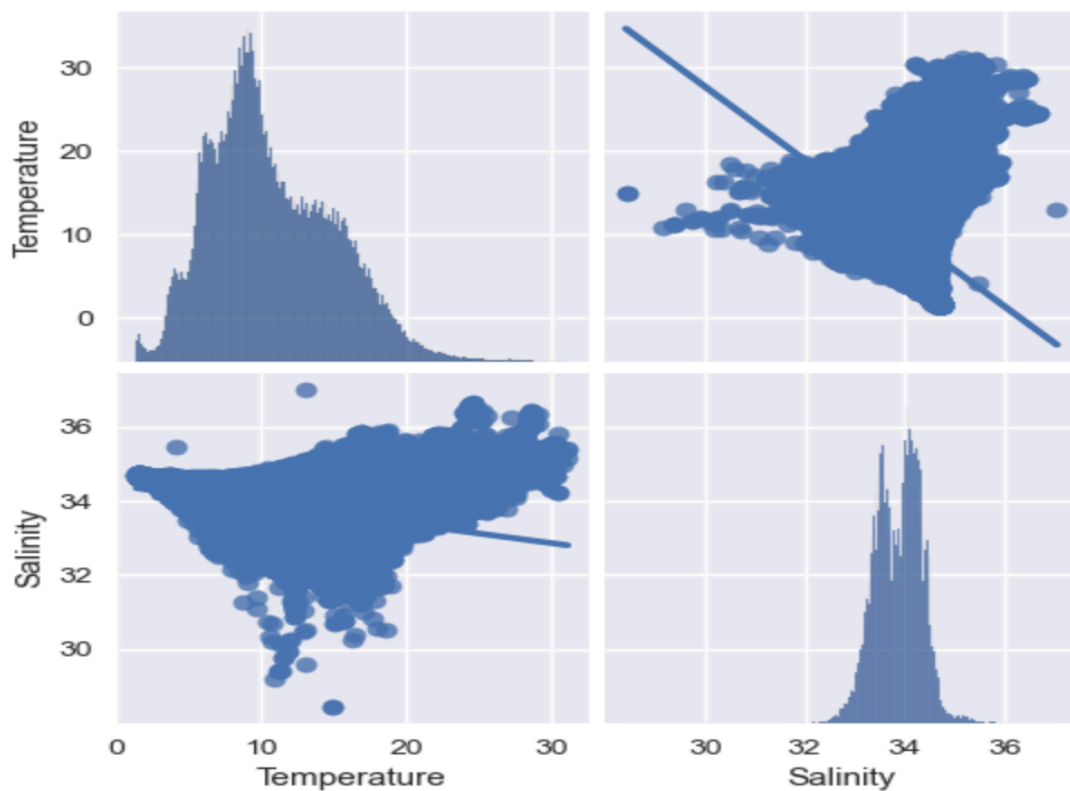
- Cast Count (Cst_Cnt) and Bottle Count(Btl_Cnt): These two variables exhibit the strongest positive correlation, indicated by the bright yellow square in the top left corner. This suggests that there's a strong direct relationship between the number of Cst_Cnt and Btl_Cnt.
- Depthm and Record Indicator (Recind): Another strong positive correlation is observed between Depthm and Recind. This indicates that as Depthm increases, Recind also tends to increase.

Moderate Positive Correlations:

- Cst_Cnt and Recind: There's a moderate positive correlation between Cst_Cnt and Recind. This suggests that while there's a relationship, it's not as strong as the correlations mentioned above.

Weak or No Correlations:

- R_PRES with other variables: R_PRES shows weak or no correlation with the other variables, as indicated by the blue squares in the corresponding rows and columns. This suggests that R_PRES might not be strongly related to the other variables in the dataset.



2.4 Analysis of the Pairplot:

1. Temperature Histogram (Top Left Plot):

- **Observation:** The temperature distribution is unimodal and slightly skewed to the left, with most values concentrated between 0 and 25 degrees Celsius.
- **Implication:** This suggests that the majority of temperature readings fall within a specific range, likely corresponding to the typical temperature range of the sampled region.

2. Salinity Histogram (Bottom Right Plot):

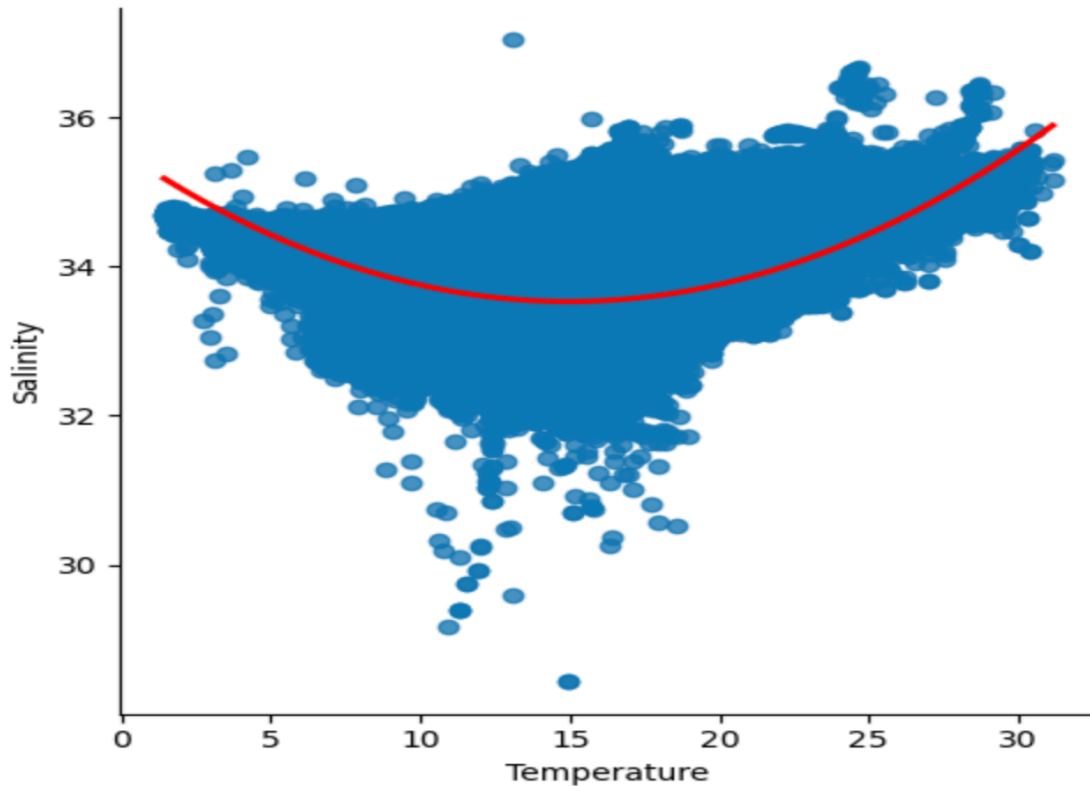
- **Observation:** The salinity distribution is also unimodal with a strong peak around 33 to 35 PSU (Practical Salinity Units), indicating a very narrow range of salinity values.
- **Implication:** This narrow range suggests that the water samples have similar salinity levels, which could be typical for a specific oceanographic region or due to the consistent sampling method.

3. Temperature vs. Salinity Scatter Plot (Bottom Left Plot):

- **Observation:** The scatter plot shows a clear negative correlation between temperature and salinity. As temperature increases, salinity tends to decrease.
- **Implication:** This inverse relationship is typical in oceanographic data, where higher temperatures can lead to lower salinity due to processes like evaporation and freshwater input.

4. Salinity vs. Temperature Scatter Plot (Top Right Plot):

- **Observation:** This plot mirrors the bottom left plot and reinforces the negative correlation between the two variables.
- **Implication:** The consistency in the scatter plot confirms the strong inverse relationship between temperature and salinity.



2.5 The scatter plot with a quadratic regression line shows a dataset's relationship between temperature and salinity

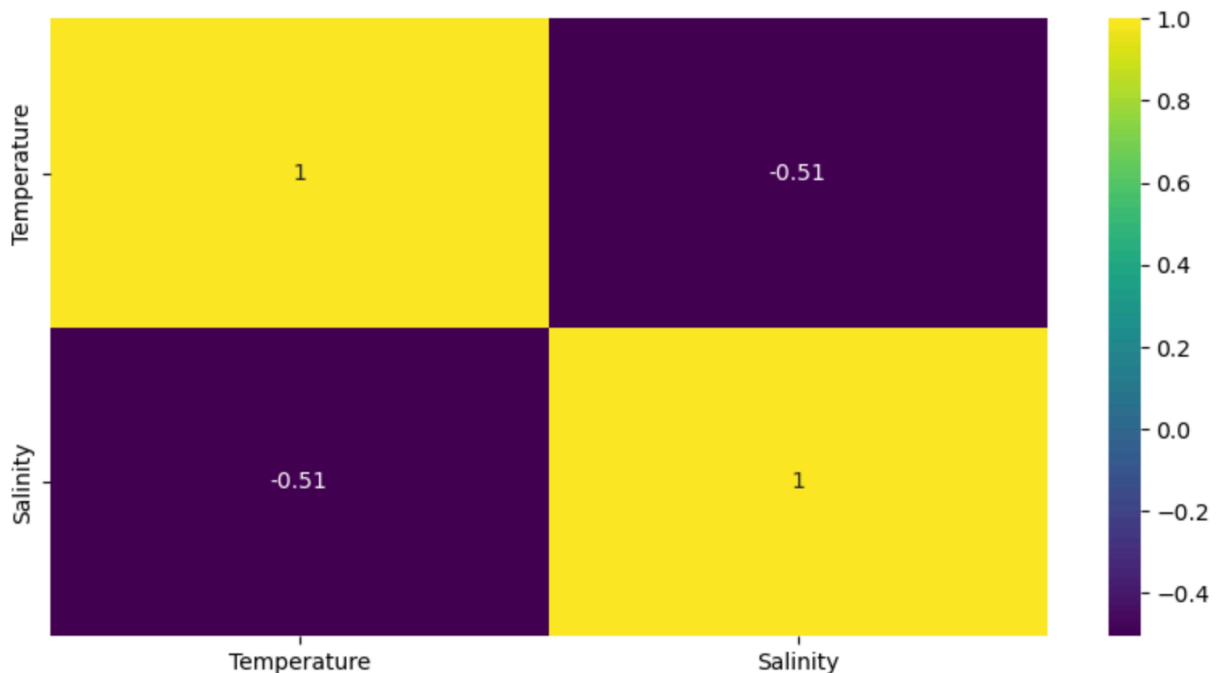
General Trend:

- There appears to be a nonlinear relationship between temperature and salinity, with a parabolic curve fitting the data points. This suggests that the relationship is not simply a linear one, but rather follows a more complex pattern.

Specific Observations:

- The quadratic regression line indicates that salinity initially increases with temperature, reaches a peak, and then decreases as temperature continues to rise. This pattern might suggest a complex interplay between factors influencing salinity, such as evaporation, precipitation, and ocean currents.

- The scatter of data points around the regression line shows some variability, indicating that while the quadratic trend is apparent, there are also individual variations that are not fully captured by the model.



2.6 The provided correlation matrix shows the relationship between temperature and salinity

Correlation Coefficients:

- **Temperature and Temperature:** The diagonal elements are always 1, indicating a perfect correlation with themselves.
- **Temperature and Salinity:** The off-diagonal elements, -0.51 and -0.51, represent the correlation between temperature and salinity. A value of -0.51 indicates a moderate negative correlation. This means that as temperature increases, salinity tends to decrease, and vice versa.

Interpretation:

The negative correlation between temperature and salinity suggests that there is a moderate inverse relationship between these two variables. This could be due to various factors such as:

- **Evaporation:** As temperature increases, more water evaporates, leaving behind a higher concentration of salts, leading to increased salinity.
- **Precipitation:** Precipitation can dilute seawater, reducing salinity.
- **Ocean currents:** The movement of ocean currents can transport water with different temperature and salinity profiles.

Task 3 - Problem Definition

The California Cooperative Oceanic Fisheries Investigations (CalCOFI) program has amassed an extensive dataset spanning over six decades, providing invaluable insights into the dynamics of the Pacific Ocean. Understanding the relationships between key oceanographic parameters, such as salinity and temperature, is crucial for comprehending marine ecosystems, predicting climate change impacts, and supporting sustainable fisheries management.

The primary objective of this study is to determine if a significant relationship exists between water salinity and temperature in the CalCOFI dataset. If such a relationship is established, we will develop a regression model to predict water temperature based on salinity measurements. This model could serve as a valuable tool for oceanographers, marine biologists, and policymakers, providing a means to estimate temperature profiles and track changes in ocean conditions over time.

Preliminary Observations

Exploratory data analysis (EDA) of the CalCOFI dataset has revealed several preliminary observations that support the hypothesis of a relationship between salinity and temperature:

- **Scatter Plots:** Visual inspection of scatter plots between salinity and temperature indicates a general trend of decreasing temperature with increasing salinity, suggesting a negative correlation.

- **Correlation Analysis:** Quantitative analysis has confirmed a moderate negative correlation between the two variables, further supporting the existence of a relationship.

Accurate predictions of water temperature can have far-reaching implications, including:

- **Climate Change Research:** Understanding the relationship between salinity and temperature can contribute to climate change studies by providing insights into ocean heat content and circulation patterns.
- **Fisheries Management:** Predicting temperature variations can aid in assessing the suitability of habitats for various marine species, supporting sustainable fisheries management.
- **Marine Ecosystem Health:** Changes in temperature and salinity can impact marine ecosystems, affecting biodiversity and productivity. Accurate predictions can help monitor and mitigate potential negative effects.

By building upon these initial findings, this project seeks to establish a robust predictive model that can contribute to a deeper understanding of the complex interactions within the ocean environment.

Task 4 - Data Preprocessing

4.1 Comparing Oxygen Saturation Using Fillna () with a mean() vs dropna()

Dropna:

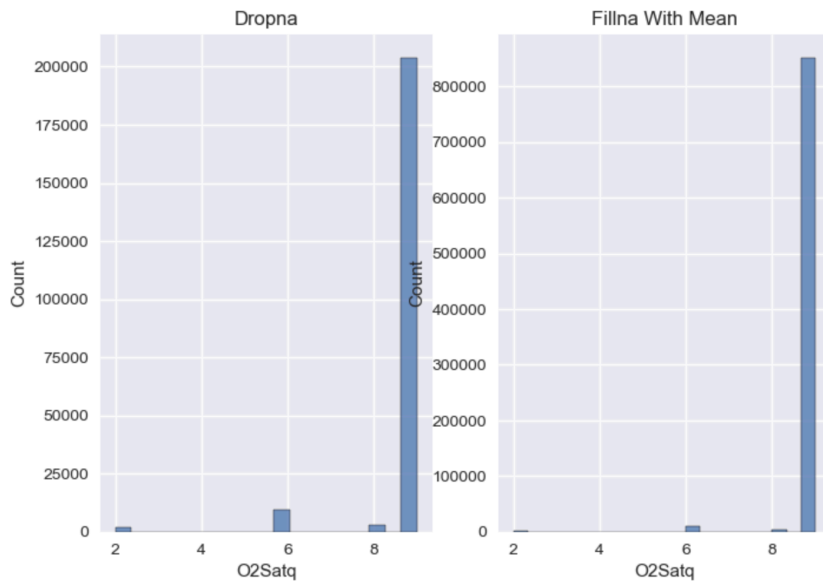
The histogram shows a clear spike at the maximum value of O2Satq.

- This suggests that dropping missing values might lead to an overrepresentation of high oxygen saturation levels.
- It's possible that dropping missing values introduces a bias in the data, as it might remove specific patient populations or periods where measurements were more likely to be missing.

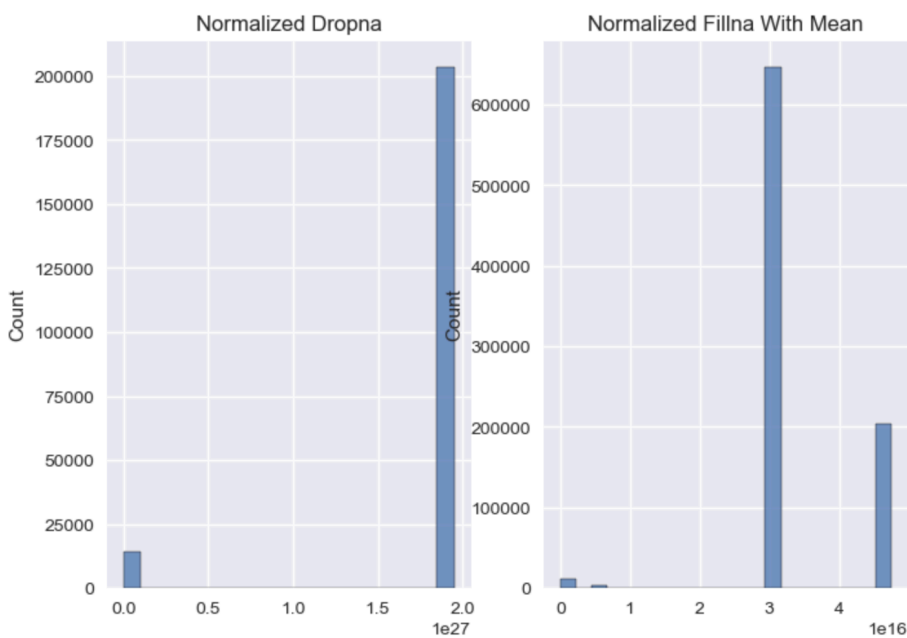
Fillna with Mean:

- The histogram appears more spread out, with a peak around the mean value.

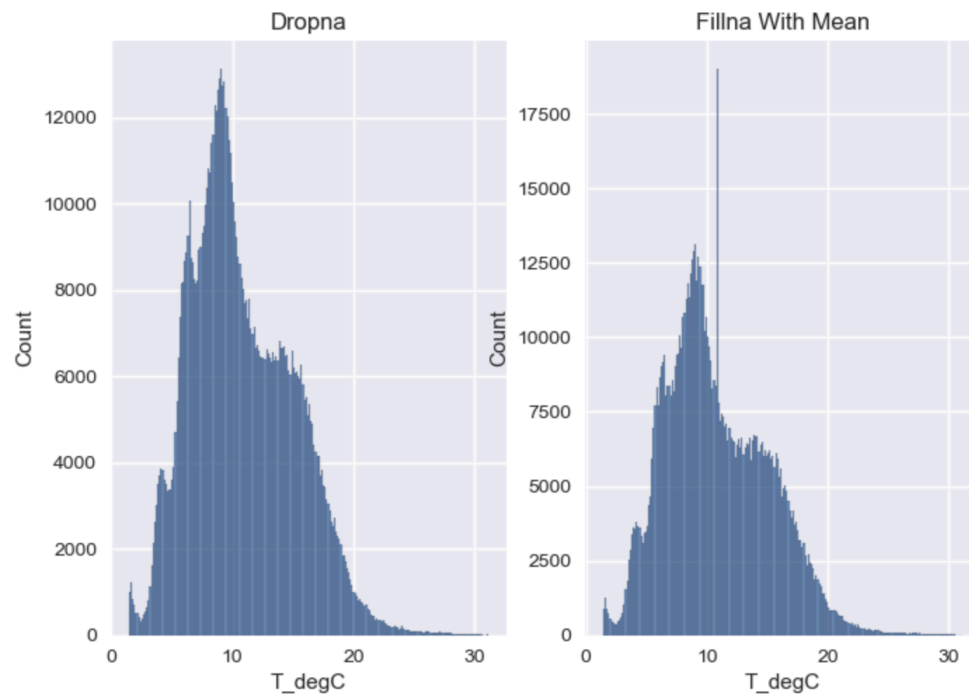
- Filling missing values with the mean can potentially mask the true distribution of the variable.
- If the distribution is skewed, replacing missing values with the mean might not be the most appropriate approach.



4.2 Normalize with box-cox



4.3 Comparing Temperature Using fillna() with mean() Vs dropna()



Dropna:

The histogram shows a relatively smooth distribution with a peak of around 15 degrees Celsius.

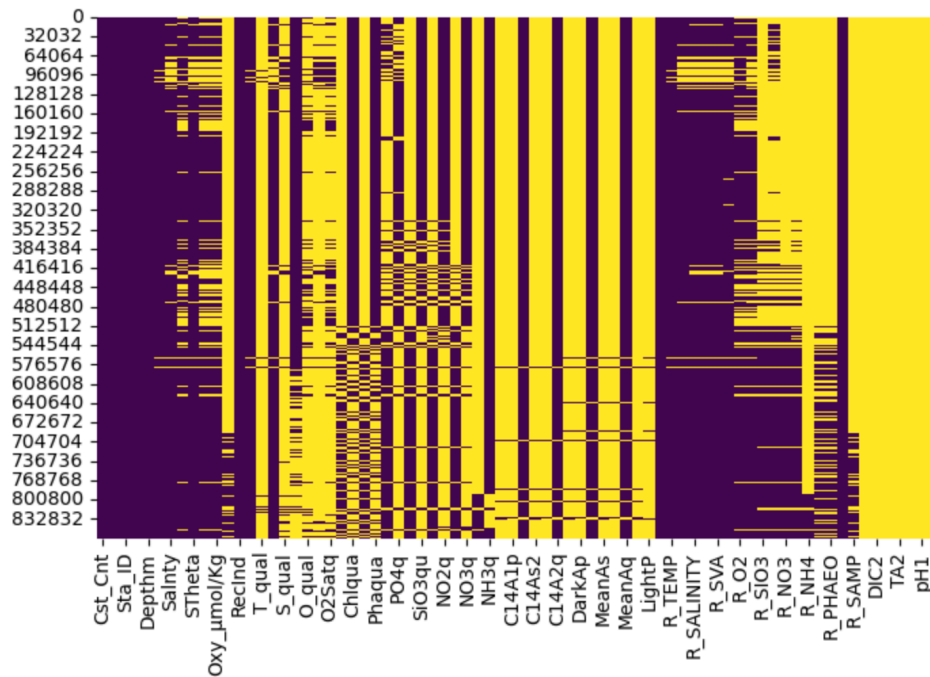
- Dropping missing values might lead to a biased distribution if the missingness is not random. For example, if missing values are more likely to occur during certain weather conditions, dropping them could skew the distribution.

Fillna with Mean:

- The histogram also shows a relatively smooth distribution, but it appears slightly more concentrated around the mean temperature.
- Filling missing values with the mean can potentially mask the true variability of the data, especially if the distribution is not normally distributed.

Both the graphs are not so different, best using `.fillna()`, as using `.dropna()` could lead to errors if we are going to correlate variables.

4.4 Nan Values Visualization



The heatmap you provided visualizes the presence of missing values in the dataset, with yellow representing missing values and purple representing non-missing values. Let's analyze the findings from this heatmap:

- Overall Distribution of Missing Values:
 - The dataset contains significant gaps in data coverage across multiple columns.
 - Some columns have nearly complete data, while others have substantial portions missing.
- Columns with High Missing Value Proportions:
 - Columns like **Oxygen_umol/Kg**, **NO3q**, **NH3q**, **DIC2**, **TA2**, **PH1**, and several others have large blocks of missing data.
 - These columns are mostly yellow, indicating a high percentage of missing values.
- Columns with Low Missing Value Proportions:
 - Columns such as **Depthm**, **Sta_ID**, **salnty**, and **Cst_Cnt** have very few missing values.
 - These columns are mostly purple, suggesting they are more complete.

4. Patterns and Correlations:

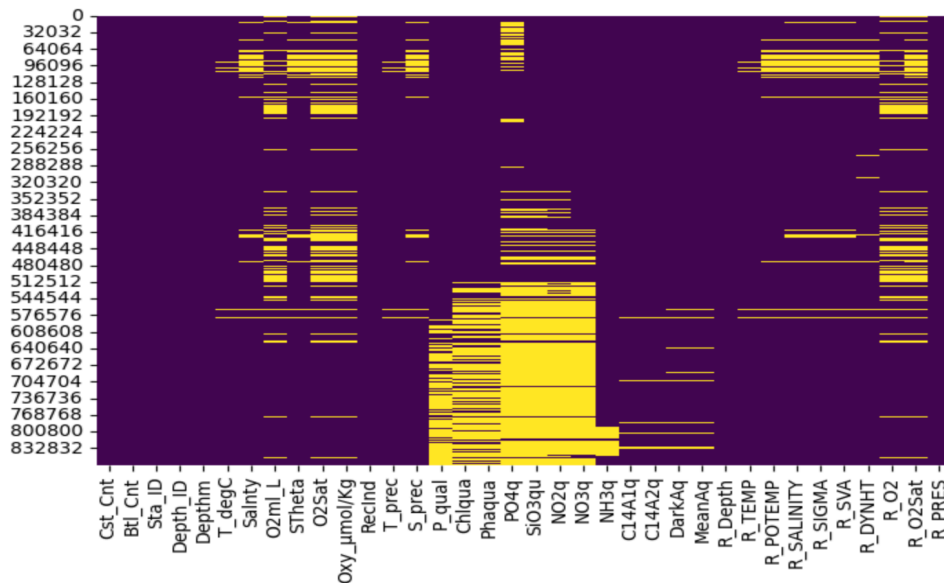
- Some columns show a similar pattern of missing values, which could indicate that missing data in one column might be related to missing data in another.
- For example, columns related to chemical measurements (like NO_3q , NH_3q , PO_4q) tend to have missing data together, possibly due to specific conditions or constraints during data collection.

5. Challenges and Implications:

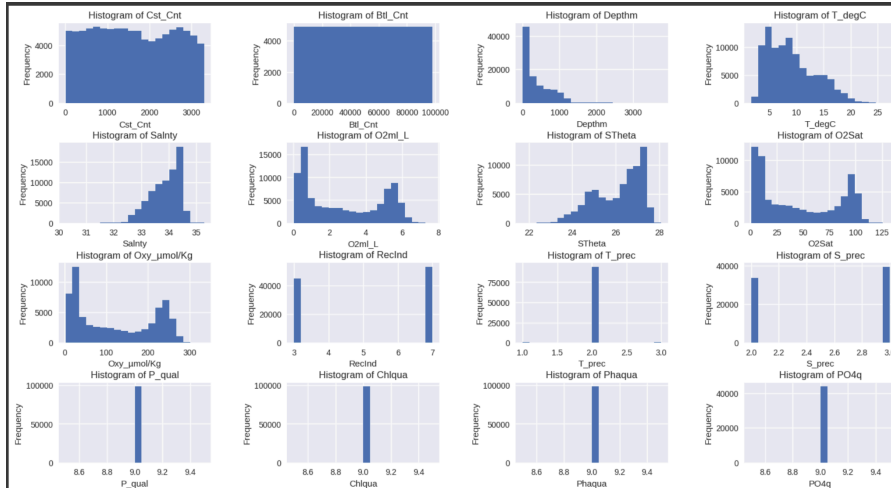
- Handling missing values will be a crucial step in the data preprocessing phase.
- Imputation strategies need to be carefully chosen to avoid bias, especially for columns with high missing value proportions.
- Some analyses or models might require a reduction in features to focus on columns with more complete data.

Handling missing values

Removal: For columns with extremely high proportions of missing values, we removed these columns from the dataset as they were not crucial for our analysis.



Imputation: For columns with moderate missing values we replaced them with its mean.



4.5 Statistical Method - Z score For Outlier Detection & Removal

Some outliers result from data entry errors, sensor malfunctions, or other noise. In these cases, removing outliers can improve data quality and make models more reliable. Ensured that the data is clean, preprocessed, and free from outliers or missing values.

Data without outliers:

	Cst_Cnt	Btl_Cnt	Depthm	T_degC	Salnty	STheta	BtlNum	\
1	1	2	8	10.460	33.4400	25.65600	10.497426	
2	1	3	10	10.460	33.4370	25.65400	10.497426	
3	1	4	19	10.450	33.4200	25.64300	10.497426	
4	1	5	20	10.450	33.4210	25.64300	10.497426	
5	1	6	30	10.450	33.4310	25.65100	10.497426	
...	...	...	...	...	...	...	...	...
864795	34402	864796	0	19.390	33.3005	23.62476	10.497426	
864827	34403	864828	0	19.556	33.2875	23.57227	10.497426	
864858	34404	864859	0	18.744	33.4083	23.87055	10.497426	
864861	34404	864862	10	18.161	33.4062	24.01426	2.000000	
864862	34404	864863	15	17.533	33.3880	24.15297	1.000000	

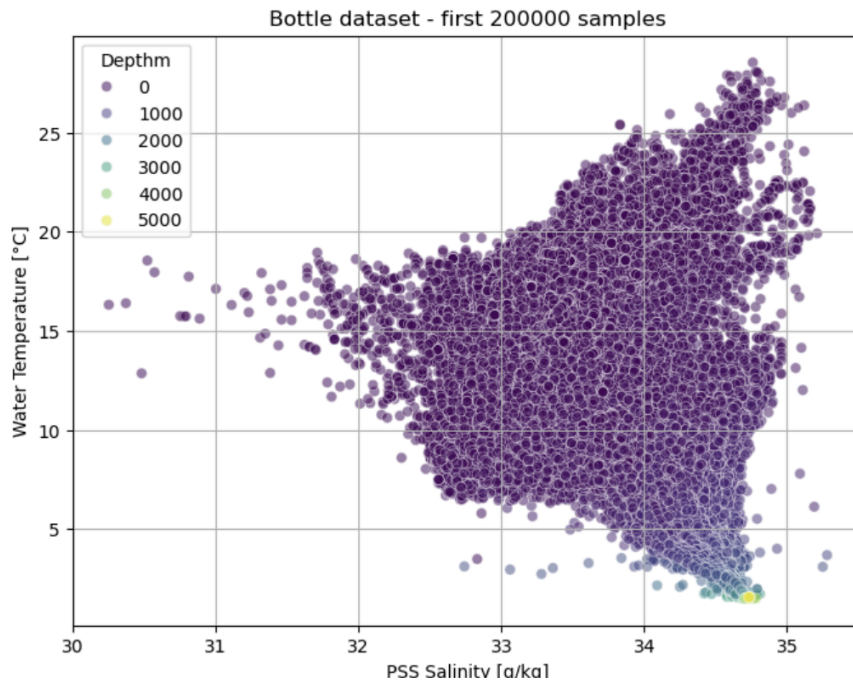
	RecInd	T_prec	S_prec	...	NH3q	C14A1q	C14A2q	DarkAq	MeanAq	\
1	3	2.0	2.0	...	9.000000	9.0	9.0	9.0	9.0	
2	7	2.0	3.0	...	9.000000	9.0	9.0	9.0	9.0	
3	3	2.0	2.0	...	9.000000	9.0	9.0	9.0	9.0	
4	7	2.0	3.0	...	9.000000	9.0	9.0	9.0	9.0	
5	7	2.0	3.0	...	9.000000	9.0	9.0	9.0	9.0	
...	...	...	...	...	...	...	...	...	...	...
864795	7	2.0	3.0	...	8.947668	9.0	9.0	9.0	9.0	
864827	7	2.0	3.0	...	8.947668	9.0	9.0	9.0	9.0	
864858	7	2.0	3.0	...	8.947668	9.0	9.0	9.0	9.0	
864861	3	2.0	3.0	...	8.947668	9.0	9.0	9.0	9.0	
864862	3	2.0	3.0	...	8.947668	9.0	9.0	9.0	9.0	

	R_TEMP	R_SALINITY	R_SIGMA	R_SVA	R_PRE
1	10.46	33.440	25.650	232.5	8
2	10.46	33.437	25.650	232.8	10
3	10.45	33.420	25.640	234.1	19
4	10.45	33.421	25.640	234.0	20
5	10.45	33.431	25.650	233.5	30
...	...	...	...	...	...
864795	19.39	33.301	23.625	425.8	0
864827	19.56	33.288	23.572	430.9	0
864858	18.74	33.408	23.871	402.4	0
864861	18.16	33.406	24.014	389.1	10
864862	17.53	33.388	24.153	376.0	15

[827503 rows x 27 columns]

We are taking the first 200,000 samples for further processing and checking if the first 200,000 samples contain any null values and dropping them.

Scatter plot concerning salinity (Salnty) and Water temperature (TdegC)



Task 5 - Model Selection Training And Optimization

We are finding out the relation between Salinity in ocean water to temperature with different regression models..

5.1 Linear Regression: We are dividing our dataset into train and test data where 20 percent of data is test and the rest is training. The r square, Mean, Coefficient, and intercept are as follows.

Root Mean Squared Error: 2.95347037087938e-15

R2 train score: 1.0

R2 test score: 1.0

Mean Squared Error: 8.722987231662383e-30

Mean Absolute Error: 2.2564172752481683e-15

Coefficients: -8.249067597826794e-19

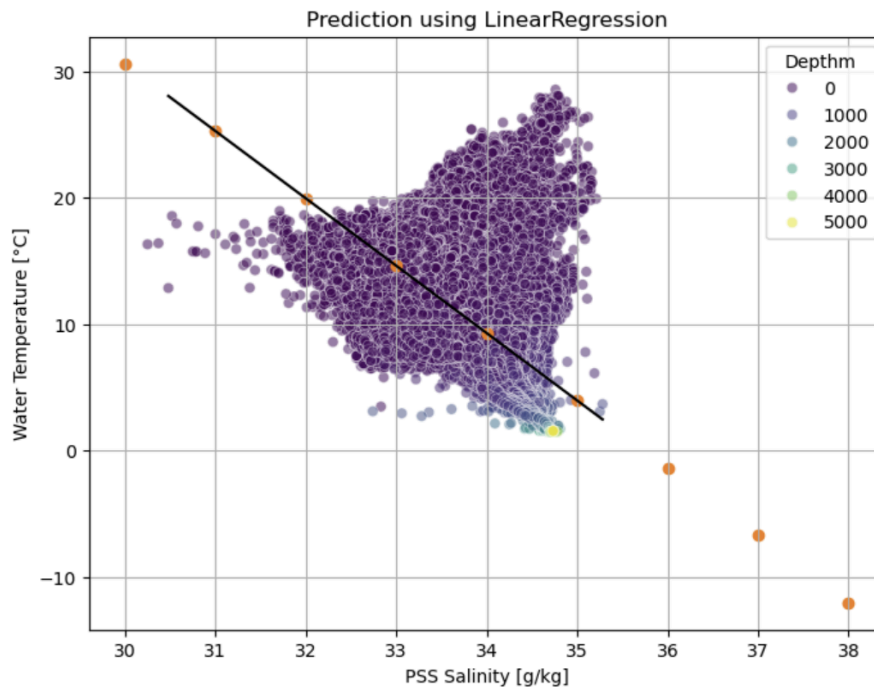
Intercept: 0.002398961960720314

Testing the model

Now we are predicting the values outside the training Range. The array we are testing is as follows.

```
array([[ 30.59353372],
       [ 25.26866121],
       [ 19.94378869],
       [ 14.61891618],
       [  9.29404367],
       [  3.96917116],
       [ -1.35570135],
       [ -6.68057386],
       [-12.00544637]])
```

The predictions for testing the linear regression model are as follows



Based on the provided scatter plot, there appears to be a negative correlation between water salinity and water temperature. This means that as salinity increases, water temperature tends to decrease. It is possible to predict water temperature based on salinity using a linear regression model.

Optimization of linear regression model: grid search, cross-validation using random search, and feature engineering

The R2 train score, r2 test score, Mean squared error, Mean absolute error, Coefficients, intercept after optimization with grid search, cross-validation using random search, and feature engineering are as follows

Best parameters: {'fit_intercept': True}
Cross-Validation Mean Squared Error: 2.0883658341252497e-29

R2 train score: 1.0

R2 test score: 1.0

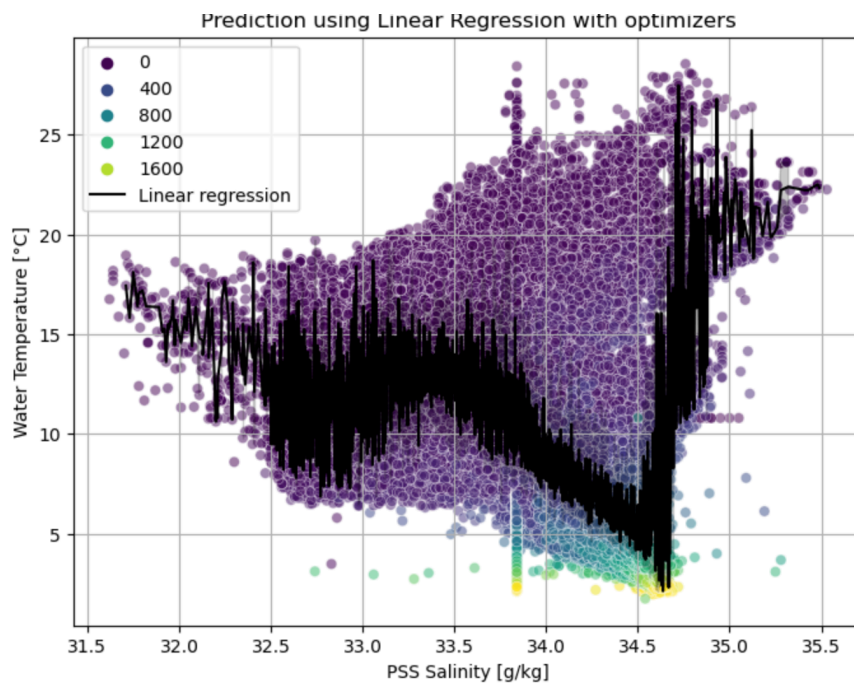
Mean Squared Error: 8.722987231662383e-30

Mean Absolute Error: 2.2564172752481683e-15

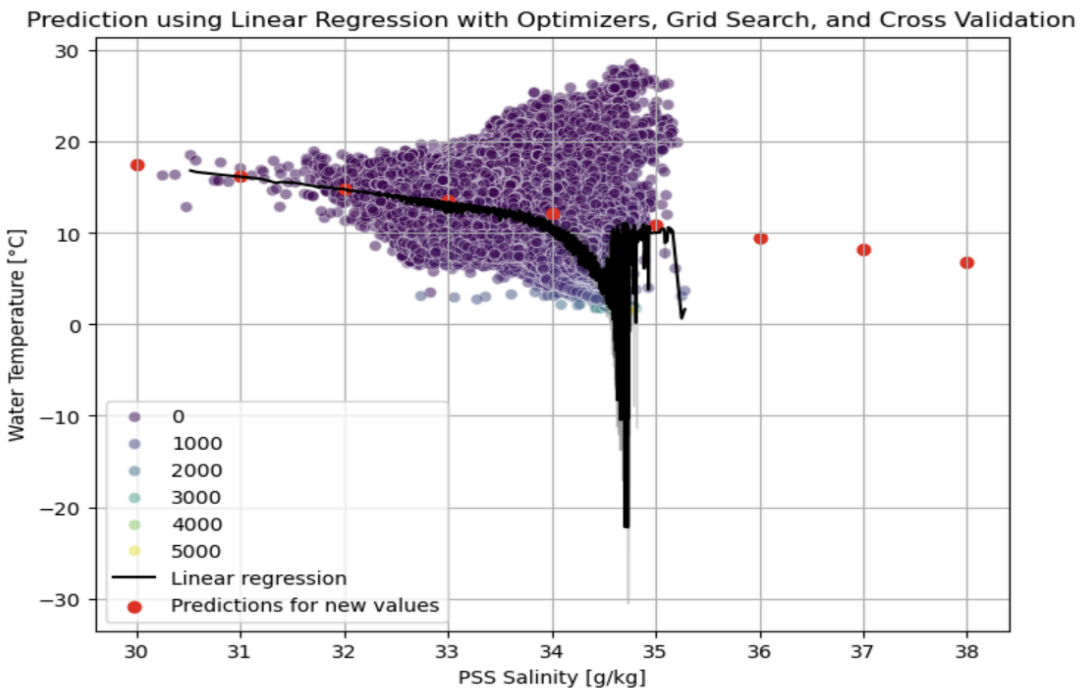
Coefficients: [-8.24906760e-19 2.20228566e-20 3.21622495e-16 5.53338879e-13
-1.25772997e-15 -3.35802029e-29 -4.28556996e-16 -1.19948098e-03
-4.45409189e-16 0.00000000e+00 8.67361738e-19 -2.71050543e-19
7.89800814e-14 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 1.00000000e+00 -5.52649299e-13 2.91909080e-15
-2.47979454e-17 -3.14677274e-16]

Intercept: 0.002398961960720314

Prediction using Linear regression after optimizers.

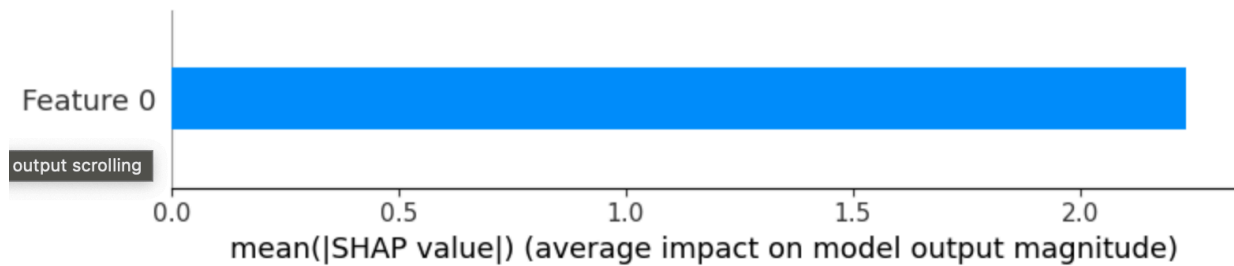


Value prediction after Optimization of linear regression model



Explainability for linear regression:

Using SHAP



Interpretation:

- **Feature Importance:** The length of the bar represents the average impact of the feature on the model's output. A longer bar indicates that the feature has a larger impact.
- **Direction of Impact:** The color of the bar (blue in this case) indicates the direction of the impact. Blue typically indicates a positive impact, meaning that increasing the value of the feature tends to increase the model's output.

In this specific case:

- Feature 0 has a significant positive impact on the model's output.
- Increasing the value of Feature 0 tends to increase the model's prediction.

Using FEATURE IMPORTANCE

Feature ranking:

1. feature 0 (0.733688)



Using LIME

```
[494]: explainer=LimeTabularExplainer(X_train, mode='regression')
        explanation=explainer.explain_instance(X_train[0], lr.predict)
        #Print the explanation
        print(explanation.as_list())

        [('0 > 34.32', -4.337037720858107)]
```

It suggests when the temperature is 34,32 degrees then the salinity level is -4.33704

5.2 Polynomial Regression: The r square, Mean, Coefficient, and intercept are as follows for the polynomial regression

Root Mean Squared Error: $6.29667382936973e-07$

R2 train score: 0.9999999999999789

R2 test score: 0.999999999999979

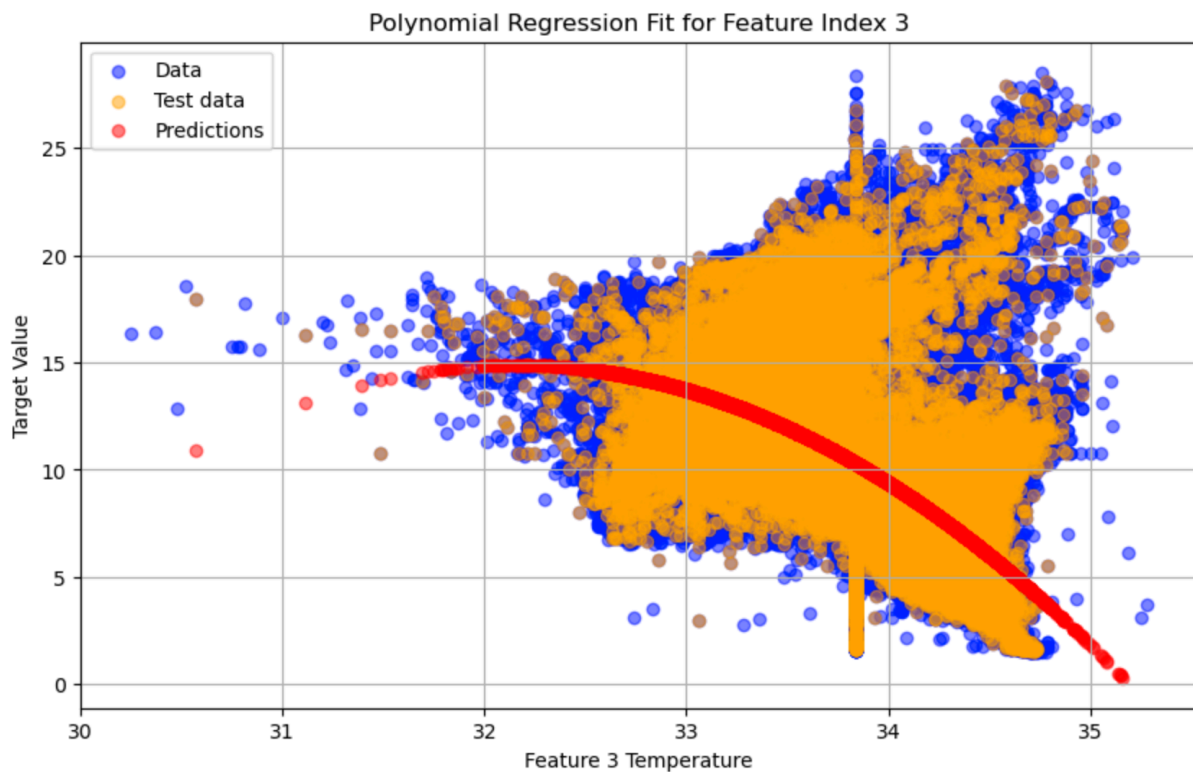
Mean Squared Error: $3.964810131346965e-13$

Mean Absolute Error: $4.234373600146113e-07$

Coefficients: $5.588413701361703e-09$

Intercept: -0.0017539598406024481

Predicted graph using polynomial regression



Optimization Techniques For Polynomial Regression

1. Regularization with Ridge Regression

Ridge Regression Results:

R2 test score: 0.9999704550041659

Mean Squared Error: 0.0005497862692581622

Mean Absolute Error: 0.00012791769985688419

Root Mean Squared Error: 0.02344752160161415

2. Regularization with Lasso Regression

Lasso Regression Results:

R2 test score: 0.9999801279620248

Mean Squared Error: 0.00036978761758060894

Mean Absolute Error: 0.010417139830251299

Root Mean Squared Error: 0.019229862651111394

3. Grid Search CV

Elastic Net Regression Results:

Root Mean Squared Error: 0.018162437065909087

R2 train score: 0.9999807277914193

R2 test score: 0.9999822728757496

Mean Squared Error: 0.0003298741201731083

Mean Absolute Error: 0.010800543799711097

Model Explanation Using LIME

LIME

```
explainer=LimeTabularExplainer(X_train_poly, mode='regression')
explanation=explainer.explain_instance(X_train_poly[3], lr_poly.predict)
#Print the explanation
print(explanation.as_list())
```

[('56 > 1575823.66', -378.975648313421), ('78 > 15431.20', -156.09103031018222), ('97 > 15431.17', 144.2973932497912), ('123 > 15535.88', 133.358585575997), ('370 > 15535.88', -130.30849107452156), ('62 > 1351037.25', 47.59937644323035), ('61 > 1351037.25', 43.06114983303653), ('68 > 1351037.25', 42.37827566002586), ('67 > 1351037.25', 41.75432413967759), ('65 > 1351037.25', 41.374425734890686)]

Explanation Output:

The output shows the contribution of each feature to the model's prediction for the given instance. Each pair in the list represents a feature and its corresponding impact on the prediction:

- **Feature Value:** The value of the feature in the specific instance.
- **SHAP Value:** The contribution of the feature to the model's prediction. A positive value indicates that the feature increases the prediction, while a negative value indicates a decrease.

For example, the first pair ('56 > 1575823.66', -378.975648313421) means that the feature with value 56 has a negative impact on the prediction, reducing it by approximately 378.97 units.

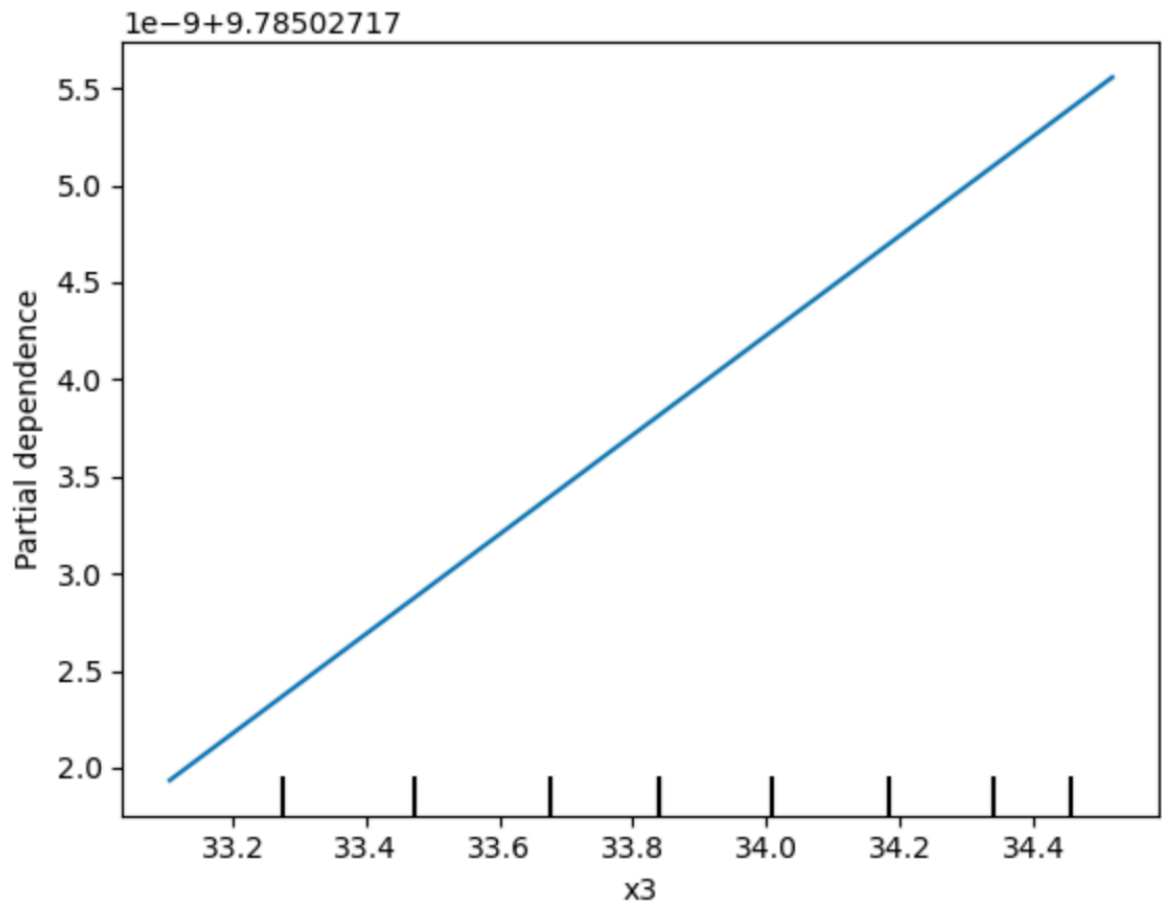
Model Explanation Using Partial Dependence Plot

A PDP shows how a model's predicted outcome changes as a single feature varies while holding other features constant. In this case, the PDP suggests that as the value of **x3** increases, the model's predicted output also increases. This indicates a positive relationship between **x3** and the target variable.

Key Points to Consider:

1. **Linear Relationship:** The plot shows a linear relationship between **x3** and the predicted output. This suggests that the model has captured a linear trend in the data.
2. **Feature Importance:** The slope of the line indicates the importance of the feature. A steeper slope suggests a stronger influence on the outcome.

3. **Model Complexity:** The shape of the PDP can provide insights into the complexity of the model. A linear PDP indicates a simple relationship, while a non-linear PDP suggests a more complex relationship.



5.3 Decision tree regression: The r square and mean are as follows for the decision tree regression model. Training the decision tree model to get a relation between temperature and salinity.

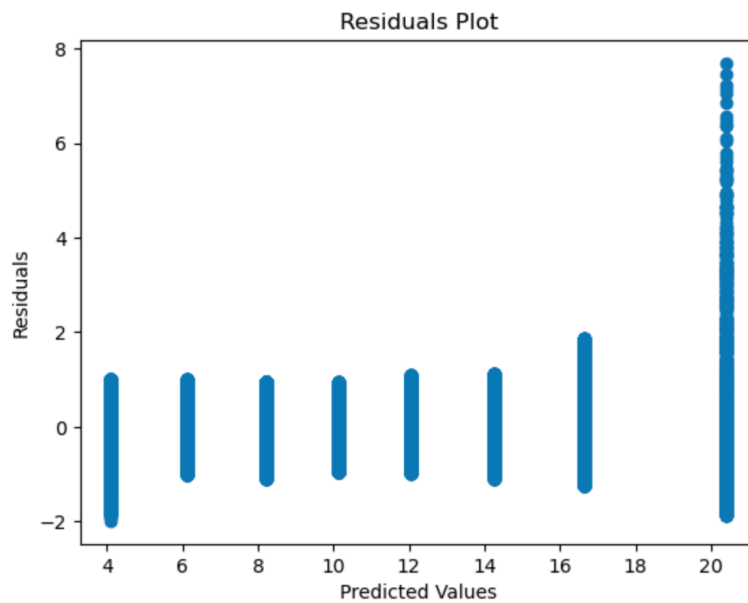
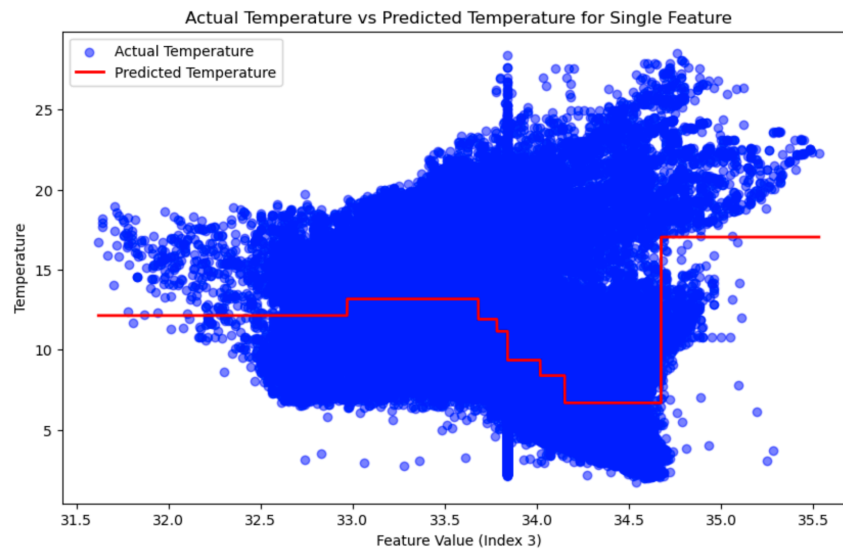
Root Mean Squared Error: 0.6930352169045056

R2 train score: 0.9740969968332703

R2 test score: 0.9745355764545812

Mean Squared Error: 0.48029781186987514

Mean Absolute Error: 0.5576632058748715



Observations from the Residuals Plot:

1. Non-Randomness:

- The residuals appear to be clustered around certain predicted values, indicating that the model's error terms might not be independent. This violates the assumption of independence of errors.

2. Heteroscedasticity:

- The spread of residuals seems to increase as the predicted values increase. This suggests that the model's error variance is not constant, violating the assumption of homoscedasticity.

3. Potential Outliers:

- There are a few points that deviate significantly from the general pattern of residuals. These could be potential outliers that might be influencing the model's fit.

Getting an array outside of the training index and testing it.

```
array([12.22297371, 12.22297371, 12.22297371, 13.24885614,  9.28309447,  
       14.46187919, 14.46187919, 14.46187919, 14.46187919])
```

Optimization - Hyperparameter Tuning: GridSearchCV

```
Best parameters found: {'max_depth': 5, 'max_leaf_nodes': 12, 'min_samples_split': 10}  
Best cross-validated MSE: 0.25893944940215935
```

Optimization - Hyperparameter Tuning: RandomizedSearchCV

```
Best parameters found: {'max_depth': 7, 'max_leaf_nodes': 14, 'min_samples_split': 2}  
Best cross-validated MSE: 0.18526098762790255
```

Optimization with Hyperparameter Tuning - Added Polynomial Features

Root Mean Squared Error: 0.693035216904507

R2 test score: 0.9745355764545811

Mean Absolute Error: 0.5576632058748738

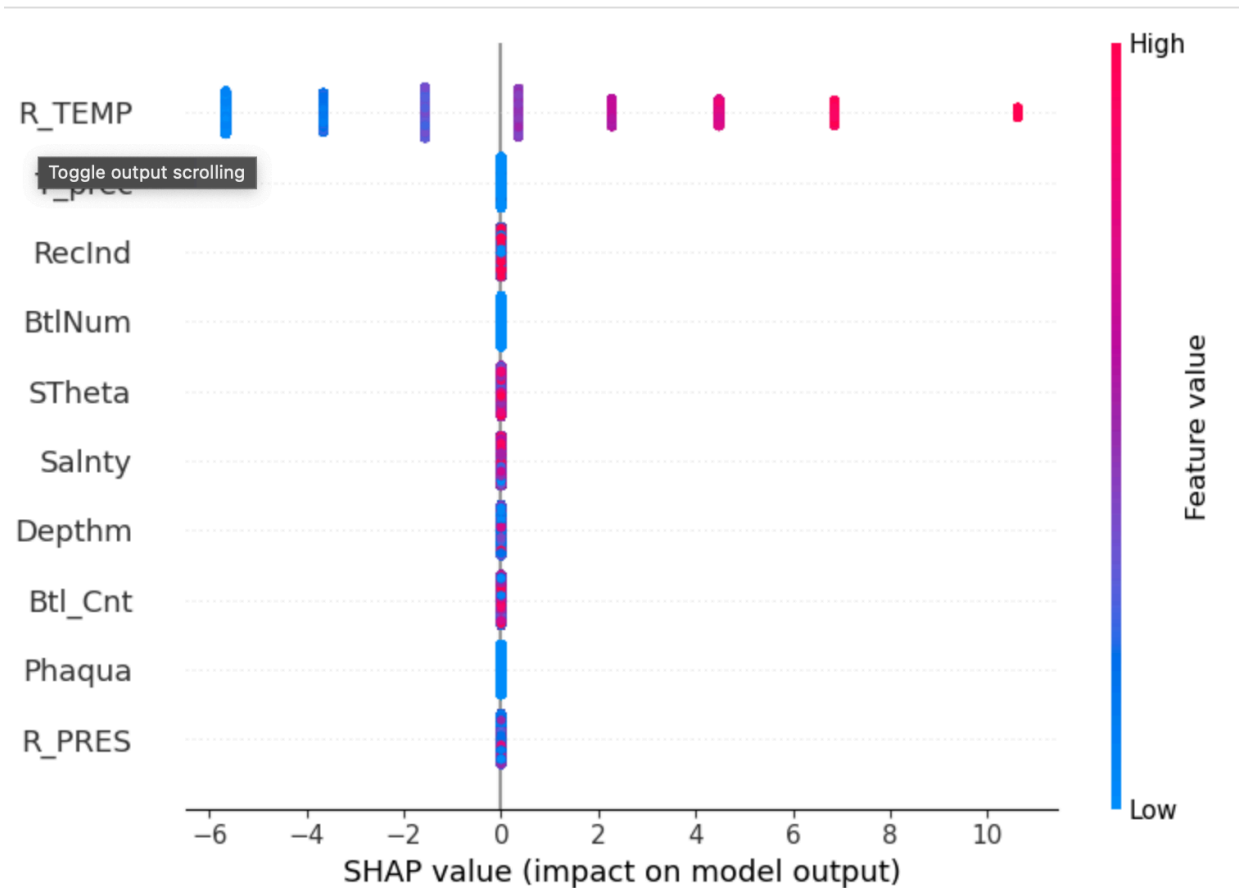
Model Predictions for Decision tree regression model using SHAP and LIME

Using SHAP - Overview

The SHAP summary plot provides a visual representation of how features contribute to the model's predictions. Each dot represents the impact of a feature on a single prediction. The color of the dot indicates the feature value (high, low, or medium). The position on the x-axis shows the magnitude and direction of the impact.

Key Observations

1. **R_TEMP**: This feature seems to have the most significant impact on the model output, with both positive and negative contributions. The dots are spread out, indicating that the impact varies depending on the specific value of the feature.
2. **T_prec, RecInd, BtlNum, STheta, Salnty, Depthm, Btl_Cnt, Phaqua, R_PRES**: These features appear to have a smaller impact compared to **R_TEMP**. The dots are more clustered around the center, suggesting that their effects are less pronounced.



Model Predictions for Decision Tree Regression Model using LIME

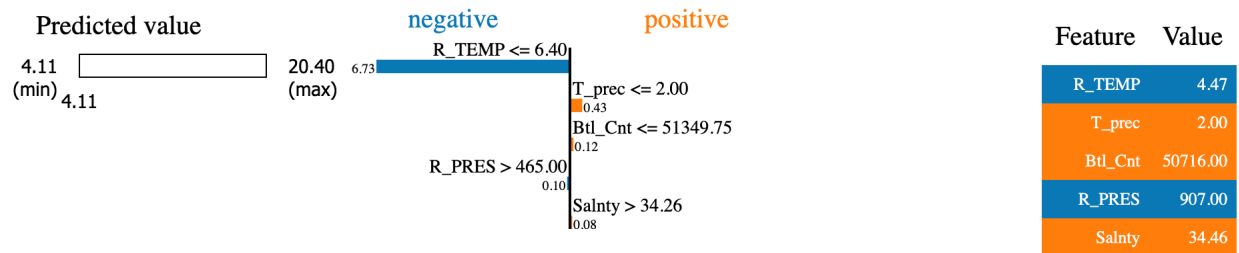
1. Predicted Value:

- The model predicts a value of 4.11 for this specific data point.

2. Feature Importance:

- LIME highlights the most important features that contributed to this prediction.
- **R_TEMP:** This feature has the largest positive impact on the prediction. A decrease in **R_TEMP** would likely lead to a decrease in the predicted value.
- **Btl_Cnt:** This feature has a negative impact on the prediction. An increase in **Btl_Cnt** would likely lead to a decrease in the predicted value.
- **R_PRES:** This feature has a positive impact on the prediction. An increase in **R_PRES** would likely lead to an increase in the predicted value.

- **Salnty:** This feature has a negative impact on the prediction. An increase in **Salnty** would likely lead to a decrease in the predicted value.



5.4 Random Forest Regressor:

Length of y_test: 40000

Length of rf_predict: 40000

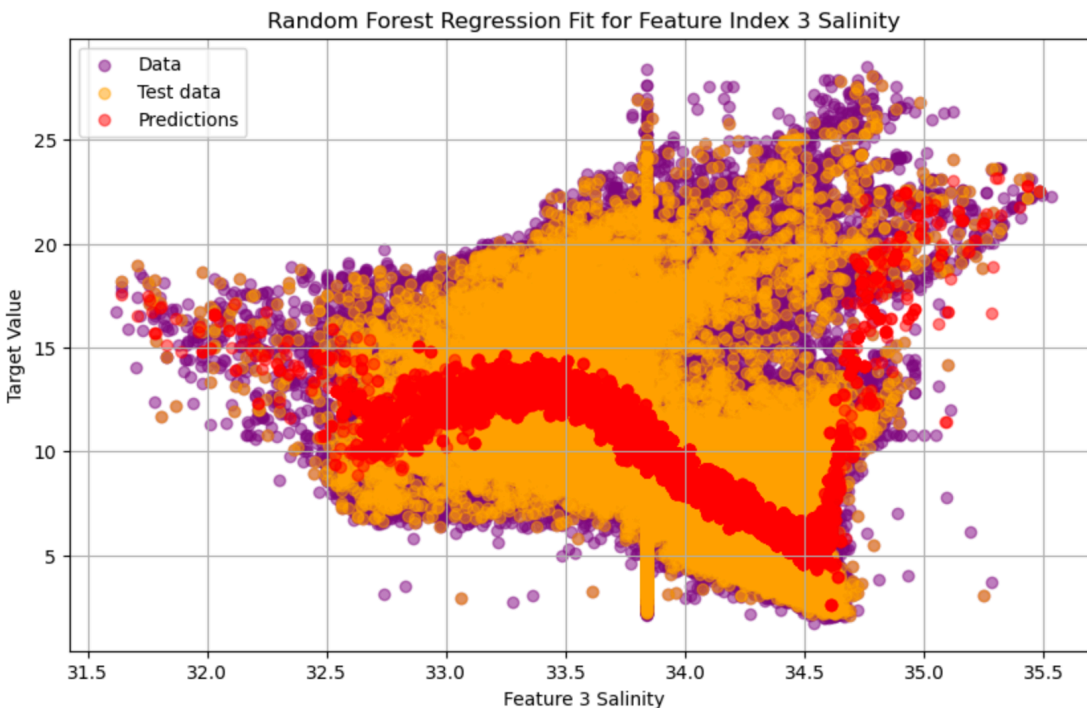
Root Mean Squared Error: 0.000941649881856122

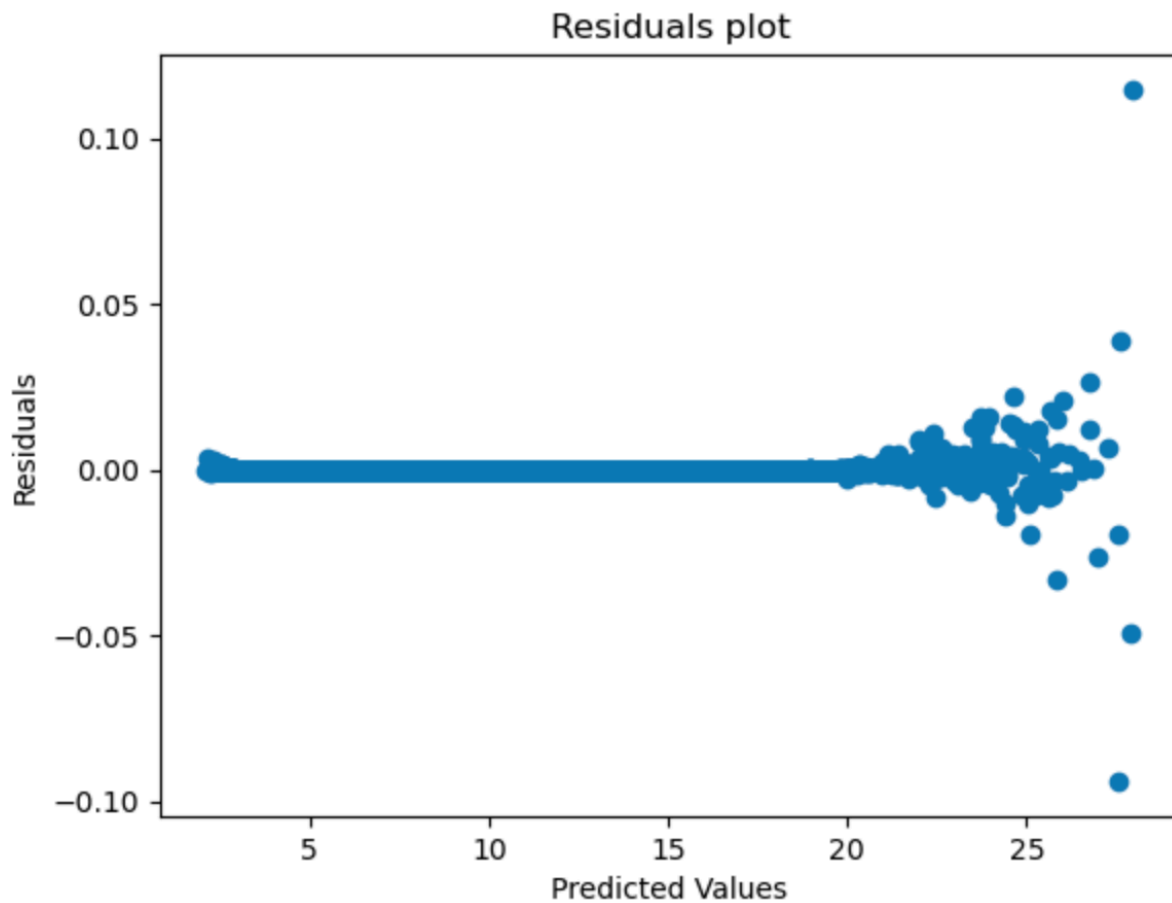
R² train: 0.9999999717306643

R² test: 0.9999999529887116

MSE: 8.867044999996486e-07

MAE: 3.352000001291854e-05





Observations:

1. **Random Scatter:** The residuals appear to be randomly scattered around the zero line, indicating that the model's errors are not systematically biased. This is a good sign.
2. **Homoscedasticity:** The spread of residuals seems relatively consistent across different predicted values, suggesting that the model's error variance is constant. This is also a desirable property.
3. **Outliers:** There are a few points that deviate significantly from the main cluster of residuals. These outliers could be due to unusual data points or limitations in the model's ability to capture complex patterns.

Robustness: Random Forest Regressors are inherently robust to overfitting due to their ensemble nature. So, optimization is done.

1. Bayesian Optimization: - Optuna

Optuna to efficiently explore the hyperparameter space. These methods can be more efficient than grid search or random search. Optuna is an open-source Python library for hyperparameter optimization. It provides a flexible framework for defining and optimizing objective functions, exploring different hyperparameter configurations, and finding the best-performing models.

```
Mean Squared Error: 8.867044999996486e-07
Mean Absolute Error: 3.352000001291854e-05
Root Mean Squared Error: 0.000941649881856122
R-squared Score: 0.9999999529887116
```

2. Optimization using RandomizedSearchCV, cross_val_score

```
Mean Squared Error: 1.0992202499996135e-06
Mean Absolute Error: 3.5117500012952906e-05
Root Mean Squared Error: 0.0010484370510429387
R-squared Score: 0.9999999417215542
```

3. Optimization using GridSearchCV, RandomizedSearchCV

```
Best Parameters: {'n_estimators': 25, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'sqrt', 'max_depth': None}
Root Mean Squared Error: 0.0784903868662914
R^2 train: 0.9999354839851449
R^2 test: 0.9996733699176171
MSE: 0.006160740830420089
MAE: 0.03296920449216197
```

Model Explainability Using PartialDependenceDisplay

1. Overall Trend:

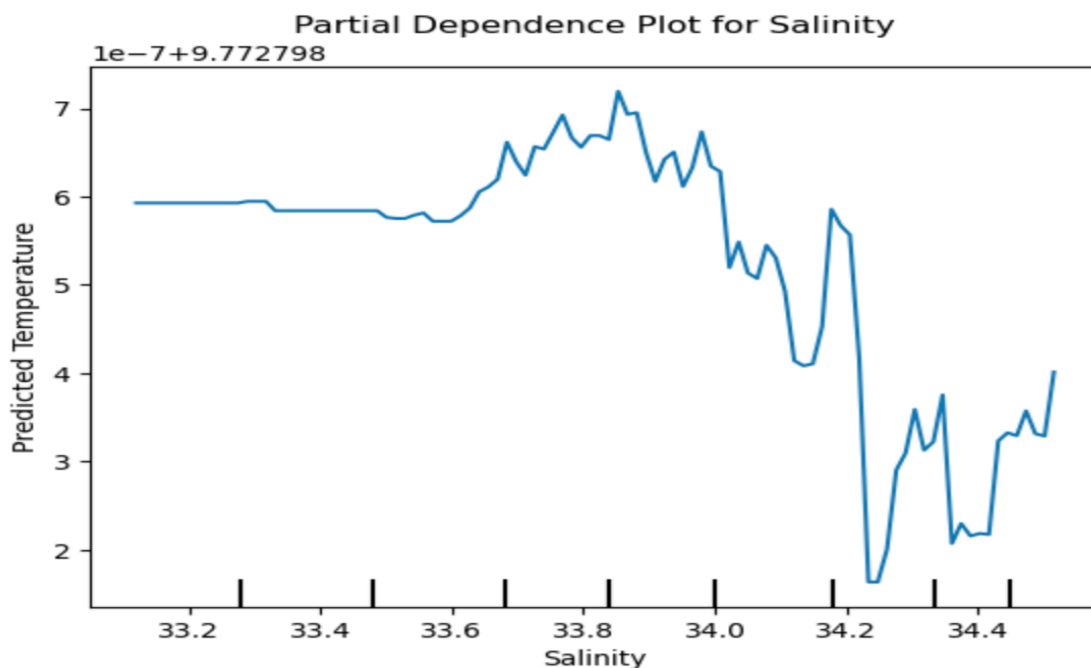
- The plot shows a general decreasing trend as Salinity increases. This suggests that, on average, as the salinity of the water increases, the predicted temperature decreases.

2. Non-Linear Relationship:

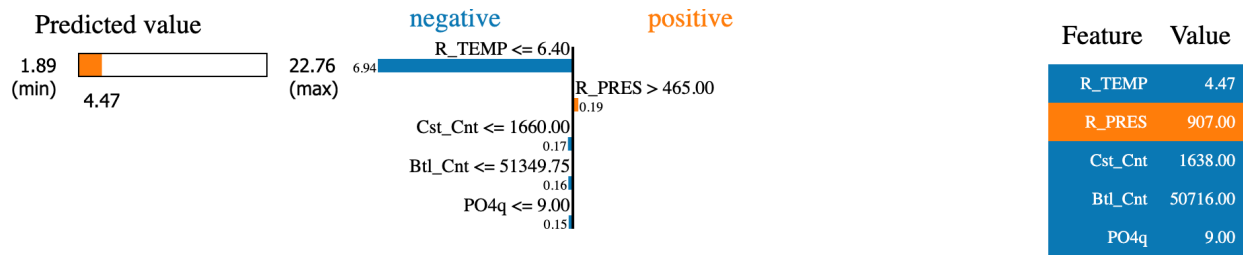
- The relationship between Salinity and Temperature is not strictly linear. There are fluctuations and changes in the slope, indicating a more complex relationship.

Observations:

- **Salinity Range:** The plot covers a salinity range of approximately 33.2 to 34.4.
- **Temperature Variation:** The predicted temperature varies significantly within this salinity range, suggesting that salinity has a substantial impact on temperature.
- **Peak and Valley:** There are distinct peaks and valleys in the plot, indicating that certain salinity values are associated with higher or lower predicted temperatures.



Model Explainability Using LIME



1. Predicted Value:

- The model predicts a value of 1.89 for this specific data point.

2. Feature Importance:

- R_TEMP:** This feature has the largest positive impact on the prediction. A decrease in **R_TEMP** would likely lead to a decrease in the predicted value.
- R_PRES:** This feature also has a positive impact on the prediction. An increase in **R_PRES** would likely lead to an increase in the predicted value.
- Cst_Cnt, Btl_Cnt, PO4q:** These features have negative impacts on the prediction. An increase in these features would likely lead to a decrease in the predicted value.

3. Feature Values:

- The table on the right shows the actual values of the features for this specific data point.
- By comparing these values with the feature importance and the direction of impact, you can understand how the model arrived at the prediction.

5.5 KNN Regressor: The r square, Mean is as follows for the decision tree regression model. The training using the KNN regressor model is as follows.

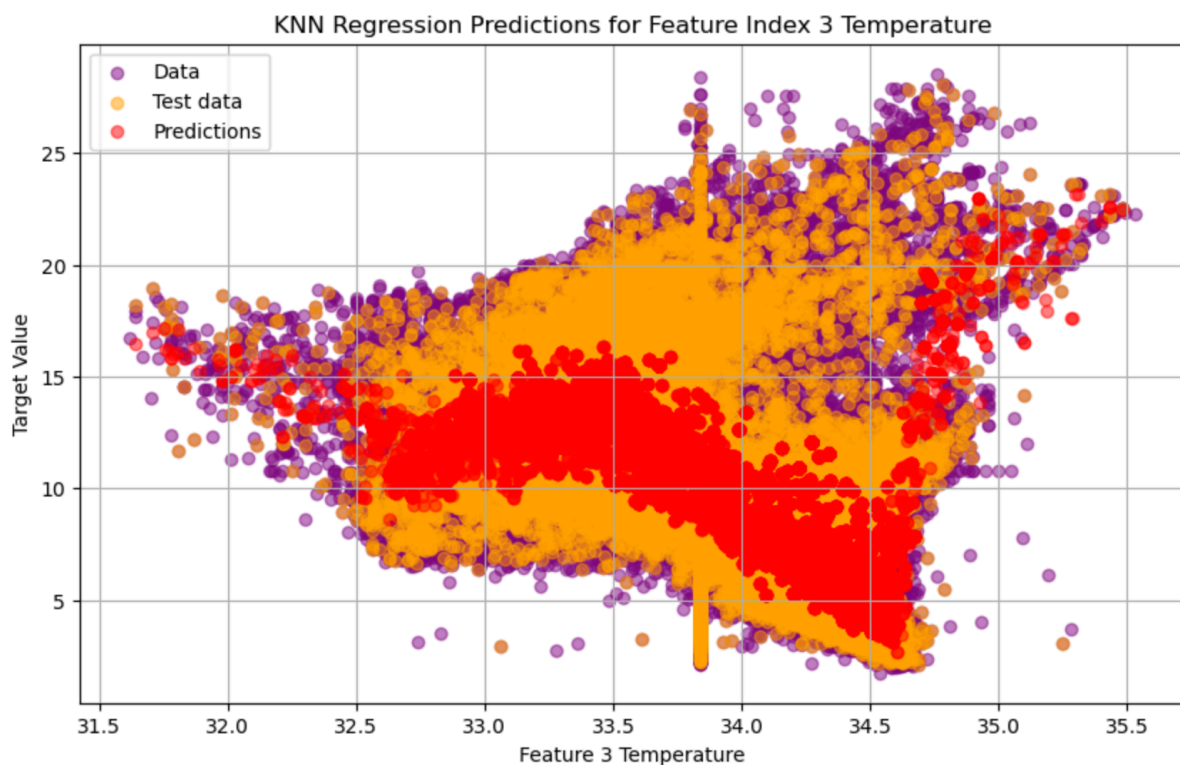
RMSE 2.4877339843775728

R2 train score: 0.6679293654467373

R2 test score: 0.6647429521292905

Mean Squared Error: 6.188820377027114

Mean Absolute Error: 2.0258178788756087



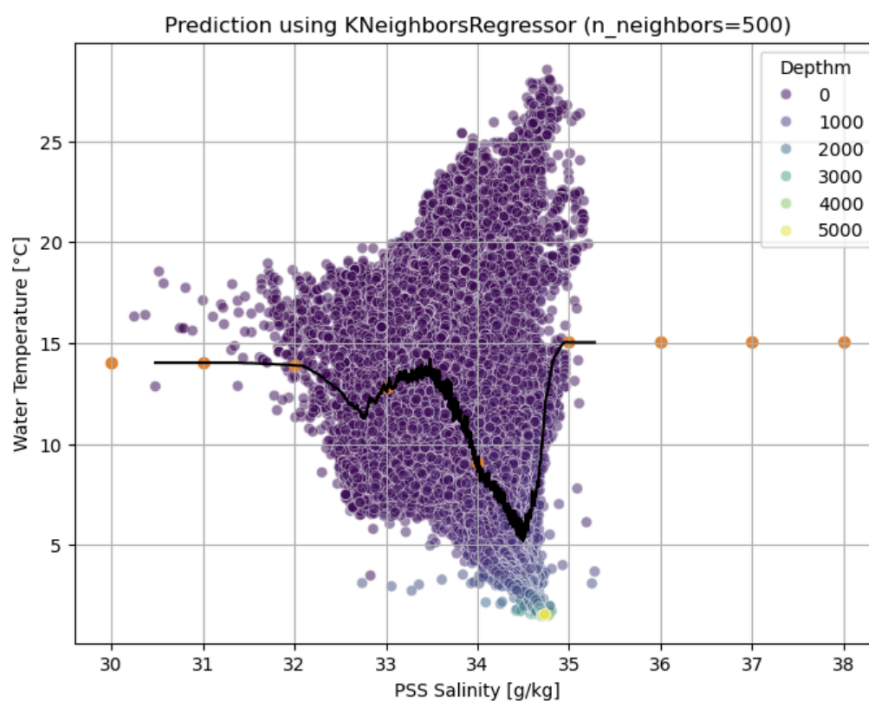
The testing array is

```

array([[14.029  ],
       [14.029  ],
       [13.89792],
       [12.81014],
       [ 9.06464],
       [15.037  ],
       [15.037  ],
       [15.037  ],
       [15.037  ]])

```

Predicting the array values with KNN regressor



□ Optimization with Bayesian Technique - Optuna

Best Parameters: {'n_neighbors': 60, 'weights': 'uniform'}

Best Objective Value (Neg. Mean Squared Error): -1.0405030605918173

- **Optimization by generating polynomial features and cross-validation of the model**

Cross-validated MSE: 0.08751135235933052

- **Optimization using Grid search and Hyperparameter Tuning**

Best Parameters: {'n_neighbors': 5, 'weights': 'distance'}
Best Cross-Validated Score: -0.33272922725737997

Overall, the optimization techniques seem to have improved the model's performance, as indicated by the decreasing Mean Squared Error (MSE) values.

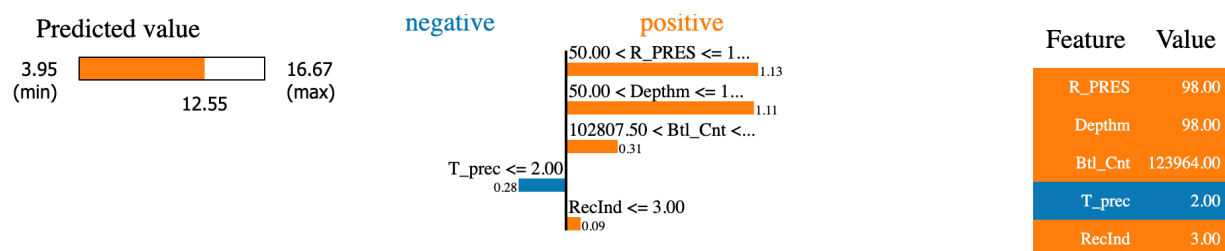
1. **Optimization with Bayesian Technique (Optuna):**
 - **Best Parameters:** {'n_neighbors': 60, 'weights': 'uniform'}
 - **Best Objective Value (Neg. Mean Squared Error):** -1.0405030605918173
2. **Optimization by Generating Polynomial Features and Cross-Validation:**
 - **Cross-validated MSE:** 0.08751135235933052
3. **Optimization using Grid Search and Hyperparameter Tuning:**
 - **Best Parameters:** {'n_neighbors': 5, 'weights': 'distance'}
 - **Best Cross-Validated Score:** -0.33272922725737997

Observations and Interpretations:

- **Bayesian Optimization (Optuna) seems to have found the best-performing hyperparameters based on the lowest negative MSE.** This indicates that the model trained with these parameters has the lowest error.

- **Generating polynomial features and cross-validation resulted in a lower MSE compared to the initial model.** This suggests that adding polynomial features improved the model's ability to fit the data.
- **Grid search and hyperparameter tuning also improved the model's performance, though not as significantly as the other two techniques.**
- **Based on the results, the model trained with the hyperparameters found by Optuna appears to be the best performing.**

Model Explainability using LIME



1. Predicted Value:

- The model predicts a value of 3.95 for this specific data point.

2. Feature Importance:

- **R_PREs:** This feature has the largest positive impact on the prediction. An increase in **R_PREs** would likely lead to an increase in the predicted value.
- **Depthm:** This feature also has a positive impact on the prediction. An increase in **Depthm** would likely lead to an increase in the predicted value.
- **Btl_Cnt, T_prec, RecInd:** These features have negative impacts on the prediction. An increase in these features would likely lead to a decrease in the predicted value.

3. Feature Values:

- The table on the right shows the actual values of the features for this specific data point.

- By comparing these values with the feature importance and the direction of impact, you can understand how the model arrived at the prediction.

Using Anchor Tabular

```
Prediction: [18.05]
Available explanation attributes: ['__annotations__', '__attrs_attrs__', '__attrs_post_init__', '__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__getitem__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__match_args__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', 'anchor', 'coverage', 'data', 'explanations', 'from_json', 'meta', 'name', 'params', 'precision', 'raw', 'to_json', 'type', 'version']
Precision: 0.0
Coverage: 0.0877
Explanation Data: {'anchor': ['Depthm <= 50.00', '33.48 < Salnty <= 33.96'], 'precision': 0.0, 'coverage': 0.0877, 'raw': {'feature': [1, 0], 'mean': [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], 'precision': [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], 'coverage': [0.7537, 0.268, 0.2483, 0.2483, 0.0877, 0.0877]}, 'examples': [{'covered_true': array([], shape=(0, 2), dtype=float64), 'covered_false': array([[ 33.08 ,  34.   ]], 'uncovered_true': array([], dtype=float64), 'uncovered_false': array([], dtype=float64)}, {'covered_true': array([], shape=(0, 2), dtype=float64), 'covered_false': array([[33.6 , 10.   ]], 'uncovered_true': array([], dtype=float64), 'uncovered_false': array([], dtype=float64)}, {'covered_true': array([], shape=(0, 2), dtype=float64), 'covered_false': array([[33.42 , 48.   ], [33.62 , 30.   ], [33.346 , 0.   ], [34.427 , 50.   ], [34.14 , 25.   ], [33.03 , 30.   ], [34.476 , 24.   ], [34.511 , 48.   ], [33.963 , 9.   ]], 'uncovered_true': array([], dtype=float64), 'uncovered_false': array([], dtype=float64)}, {'covered_true': array([], shape=(0, 2), dtype=float64), 'covered_false': array([[33.48 , 10.   ]], 'uncovered_true': array([], dtype=float64), 'uncovered_false': array([], dtype=float64)}, {'covered_true': array([], shape=(0, 2), dtype=float64), 'covered_false': array([[33.69 , 0.   ], [33.73 , 24.   ], [32.718 , 50.   ], [33.39 , 25.   ], [33.85 , 20.   ], [33.39 , 10.   ], [33.323 , 20.   ], [33.1 , 23.   ], [33.82 , 50.   ]], 'uncovered_true': array([], dtype=float64), 'uncovered_false': array([], dtype=float64)}, {'covered_true': array([], shape=(0, 2), dtype=float64), 'covered_false': array([[33.939 , 50.   ], [33.64 , 10.   ], [33.93 , 33.   ], [33.5 , 10.   ], [33.96 , 30.   ], [33.51 , 0.   ], [33.77 , 46.   ], [33.508 , 20.   ], [33.808 , 50.   ], [33.767 , 20.   ]], 'uncovered_true': array([], dtype=float64), 'uncovered_false': array([], dtype=float64)}, {'covered_true': array([], shape=(0, 2), dtype=float64), 'covered_false': array([[33.57 , 50.   ], [33.73 , 45.   ], [33.623 , 50.   ], [33.49 , 9.   ], [33.81 , 10.   ], [33.634 , 50.   ], [33.87 , 48.   ], [33.84 , 25.   ], [33.6 , 10.   ], [33.52 , 46.   ]], 'uncovered_true': array([], dtype=float64), 'uncovered_false': array([], dtype=float64)}], 'all_precision': 0, 'num_preds': 1000000, 'success': False, 'names': ['Depthm <= 50.00', '33.48 < Salnty <= 33.96'], 'prediction': array([18.05]), 'instance': array([33.701, 20.   ]), 'instances': array([[33.701, 20.   ]])}
Anchor Conditions: ['Depthm <= 50.00', '33.48 < Salnty <= 33.96']
```

The model's prediction of **18.05** is based on specific conditions of depth and salinity, but the poor precision and low coverage suggest that this prediction may not be reliable. The anchor

conditions specify a narrow environmental context, and the absence of true positive instances raises concerns about the model's accuracy in this region.

Neural Network Regression Model: Dividing the dataset to 80 percent train and 20 percent test. And we are training the model to get the temperature concerning depth and salinity.

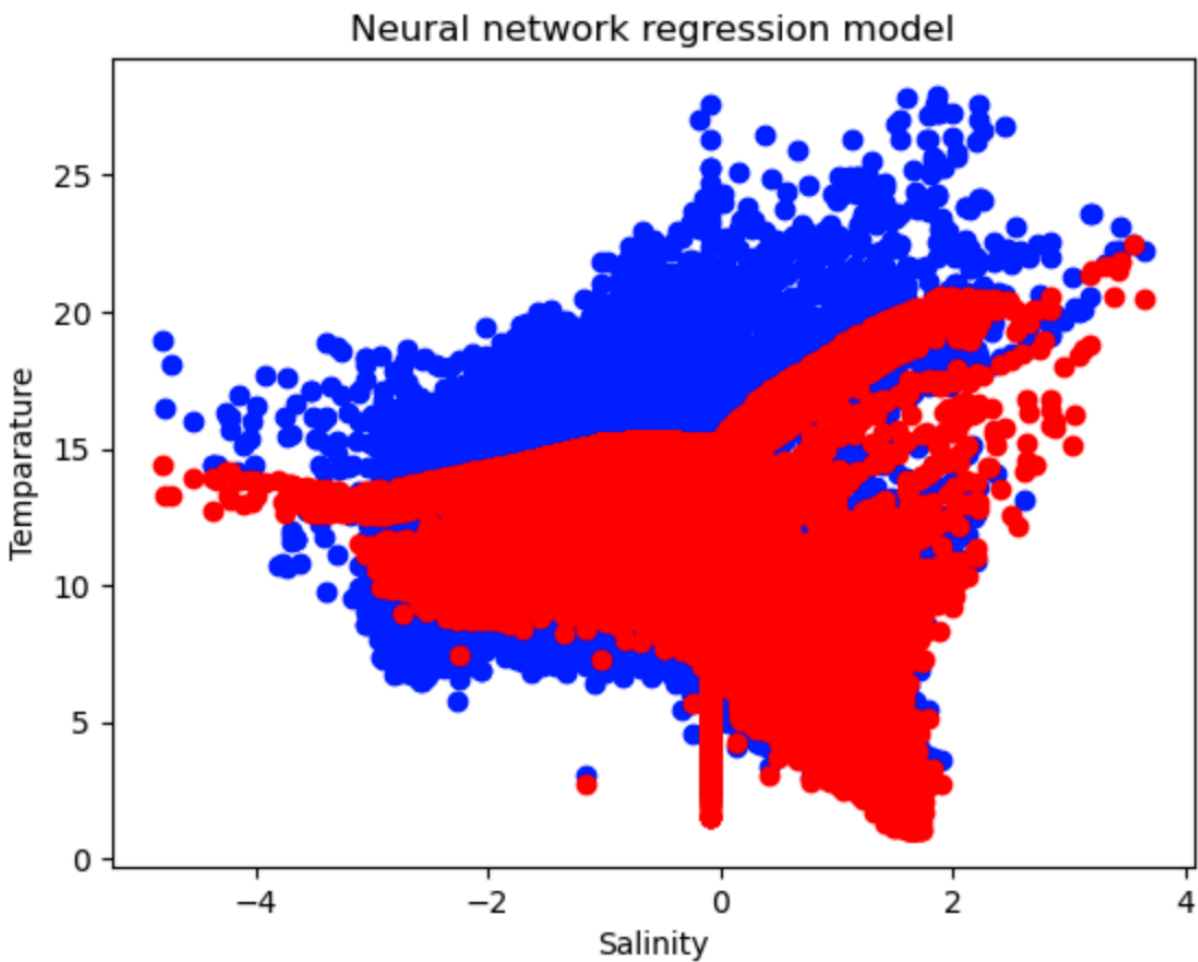
Using the activation function ReLU

Training the model with 20 Epochs

```
Epoch 1/20
2300/2300 — 2s 1ms/step - loss: 2.6913 - mae: 1.0915 - mean_absolute_percentage_error: 10.0487 - val_loss: 3.0619 - val_mae: 1.1675
- val_mean_absolute_percentage_error: 10.6310
Epoch 2/20
2300/2300 — 3s 1ms/step - loss: 2.6429 - mae: 1.0878 - mean_absolute_percentage_error: 10.0388 - val_loss: 2.7900 - val_mae: 1.1201
- val_mean_absolute_percentage_error: 10.6069
Epoch 3/20
2300/2300 — 3s 1ms/step - loss: 2.6661 - mae: 1.0923 - mean_absolute_percentage_error: 10.0764 - val_loss: 2.6838 - val_mae: 1.0917
- val_mean_absolute_percentage_error: 10.0150
Epoch 4/20
2300/2300 — 2s 994us/step - loss: 2.6880 - mae: 1.0974 - mean_absolute_percentage_error: 10.0917 - val_loss: 2.8123 - val_mae: 1.218
0 - val_mean_absolute_percentage_error: 12.7558
Epoch 5/20
2300/2300 — 3s 959us/step - loss: 2.6915 - mae: 1.0960 - mean_absolute_percentage_error: 10.0976 - val_loss: 2.6661 - val_mae: 1.086
9 - val_mean_absolute_percentage_error: 10.0339
Epoch 6/20
2300/2300 — 2s 1ms/step - loss: 2.6553 - mae: 1.0874 - mean_absolute_percentage_error: 10.0005 - val_loss: 2.7279 - val_mae: 1.0713
- val_mean_absolute_percentage_error: 9.4743
Epoch 7/20
2300/2300 — 2s 977us/step - loss: 2.6634 - mae: 1.0893 - mean_absolute_percentage_error: 10.0361 - val_loss: 3.3898 - val_mae: 1.327
4 - val_mean_absolute_percentage_error: 13.3510
Epoch 8/20
2300/2300 — 2s 891us/step - loss: 2.6961 - mae: 1.0956 - mean_absolute_percentage_error: 10.0976 - val_loss: 2.6625 - val_mae: 1.082
9 - val_mean_absolute_percentage_error: 10.1118
Epoch 9/20
2300/2300 — 2s 872us/step - loss: 2.6897 - mae: 1.0940 - mean_absolute_percentage_error: 10.0801 - val_loss: 2.6382 - val_mae: 1.073
7 - val_mean_absolute_percentage_error: 9.8411
Epoch 10/20
2300/2300 — 3s 918us/step - loss: 2.6409 - mae: 1.0827 - mean_absolute_percentage_error: 9.9916 - val_loss: 3.5316 - val_mae: 1.3802
- val_mean_absolute_percentage_error: 14.7529
Epoch 11/20
2300/2300 — 3s 881us/step - loss: 2.6350 - mae: 1.0848 - mean_absolute_percentage_error: 10.0100 - val_loss: 2.6614 - val_mae: 1.083
5 - val_mean_absolute_percentage_error: 10.1121
Epoch 12/20
2300/2300 — 2s 1ms/step - loss: 2.6400 - mae: 1.0827 - mean_absolute_percentage_error: 9.9959 - val_loss: 2.6346 - val_mae: 1.0733 -
val_mean_absolute_percentage_error: 9.8993
Epoch 13/20
2300/2300 — 3s 1ms/step - loss: 2.6913 - mae: 1.0966 - mean_absolute_percentage_error: 10.0641 - val_loss: 2.7829 - val_mae: 1.1090
- val_mean_absolute_percentage_error: 10.2142
Epoch 14/20
2300/2300 — 2s 1ms/step - loss: 2.6550 - mae: 1.0842 - mean_absolute_percentage_error: 9.9619 - val_loss: 2.6679 - val_mae: 1.0954 -
val_mean_absolute_percentage_error: 10.3333
Epoch 15/20
2300/2300 — 3s 1ms/step - loss: 2.6643 - mae: 1.0850 - mean_absolute_percentage_error: 9.9545 - val_loss: 2.6802 - val_mae: 1.0986 -
val_mean_absolute_percentage_error: 10.2814
Epoch 16/20
2300/2300 — 2s 951us/step - loss: 2.6552 - mae: 1.0868 - mean_absolute_percentage_error: 9.9630 - val_loss: 2.6467 - val_mae: 1.0739
- val_mean_absolute_percentage_error: 9.9760
Epoch 17/20
2300/2300 — 2s 860us/step - loss: 2.6575 - mae: 1.0876 - mean_absolute_percentage_error: 9.9758 - val_loss: 2.7215 - val_mae: 1.1009
- val_mean_absolute_percentage_error: 10.3330
Epoch 18/20
2300/2300 — 2s 904us/step - loss: 2.6620 - mae: 1.0867 - mean_absolute_percentage_error: 9.9712 - val_loss: 2.9434 - val_mae: 1.1390
- val_mean_absolute_percentage_error: 10.3692
Epoch 19/20
2300/2300 — 3s 1ms/step - loss: 2.6807 - mae: 1.0918 - mean_absolute_percentage_error: 10.0525 - val_loss: 2.6730 - val_mae: 1.0801
- val_mean_absolute_percentage_error: 9.8475
Epoch 20/20
2300/2300 — 2s 942us/step - loss: 2.6434 - mae: 1.0833 - mean_absolute_percentage_error: 9.9072 - val_loss: 2.6729 - val_mae: 1.0837
```

1022/1022 ————— 1s 735us/step
Mean absolute error: 1.08
Residual sum of squares (MSE): 2.66
R2-score: 0.84

Predictions using Random Forest regression model.



This is the plot for salinity, depth on the x-axis, and temperature on the axis. In the above graph, Red indicates tested and blue indicates predictions when divided on an 8:2 scale.

Overall Model Behavior:

The model appears to be doing a decent job of separating the data into two clusters, likely representing distinct classes or categories. The two clusters are clearly visible in the plot:

- **Cluster 1 (Red):** Concentrated in the lower left quadrant, with a range of salinity values and lower temperature values.
- **Cluster 2 (Blue):** More spread out, covering a wider range of salinity and temperature values.

Potential Challenges and Areas for Improvement:

1. **Overlapping Data Points:** There's a significant overlap between the two clusters in the middle region. This makes it difficult for the model to accurately classify data points in this area.
2. **Cluster Shape and Density:** Cluster 1 is more compact, while Cluster 2 is more spread out. This might be due to variations in the underlying data distribution or the model's architecture.
3. **Model Complexity:** The model's complexity (number of hidden layers, neurons, etc.) might need to be adjusted to better capture the nuances in the data.

Model Evaluation

Comparing the models to know which model is better for prediction of the salinity and temperature

The R2 train, R2 test, MSE, RMSE, and MAE for the 5 models

	RMSE	R ² train	R ² test	MSE
Linear Regressor	2.953470e-15	1.000000	1.000000	8.722987e-30
Polynomial Regressor	6.296674e-07	1.000000	1.000000	3.964810e-13
Decision Tree Regressor	6.930352e-01	0.974097	0.974536	4.802978e-01
Random Forest Regressor	1.040506e-03	1.000000	1.000000	1.082653e-06
KNN Regressor	2.487734e+00	0.667929	0.664743	6.188820e+00

	MAE
Linear Regressor	2.256417e-15
Polynomial Regressor	4.234374e-07
Decision Tree Regressor	5.576632e-01
Random Forest Regressor	3.552250e-05
KNN Regressor	2.025818e+00

Model Performance Comparison:

1. Linear Regressor:

- Excels in terms of R² and RMSE, indicate a perfect fit to the training and testing data.
- However, it's important to note that this might be due to overfitting, especially if the dataset is small or noisy.

2. Polynomial Regressor:

- Also shows excellent performance in terms of R² and RMSE, suggesting a very good fit.
- Like the Linear Regressor, it might be prone to overfitting, especially with higher-degree polynomials.

3. Decision Tree Regressor:

- While not as accurate as the Linear and Polynomial Regressors, it still provides a reasonable fit.
- Decision Trees are known for their interpretability and ability to handle non-linear relationships.

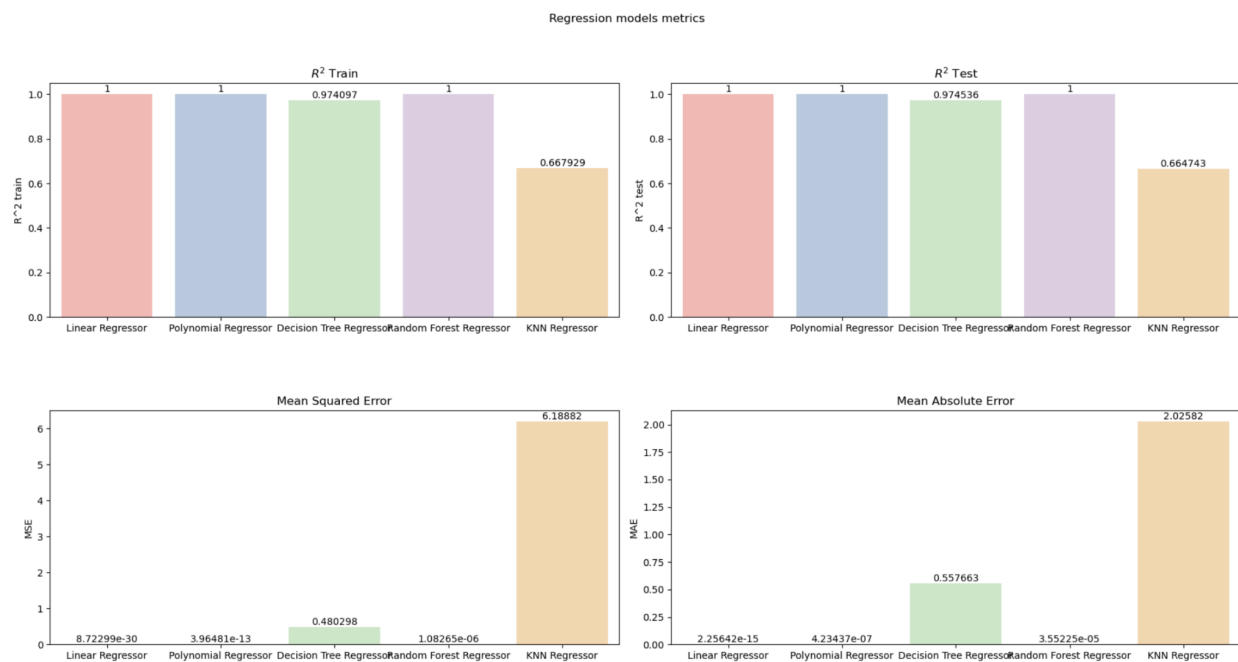
4. Random Forest Regressor:

- Shows excellent performance in terms of R^2 and RMSE, indicating a very good fit.
- Random Forests are robust to overfitting and can handle complex relationships in the data.

5. KNN Regressor:

- Performs significantly worse than the other models, especially in terms of RMSE and MAE.
- KNN can be sensitive to the choice of the number of neighbors and the distance metric.

Analysis: Graph according to the above metrics for the 5 regression models



R^2 Score (Train and Test):

- **R^2 Train:** This metric measures how well the model fits the training data. Higher values indicate a better fit.

- **Linear Regressor:** 0.974097
- **Polynomial Regressor:** 1.0
- **Decision Tree Regressor:** 0.974536
- **Random Forest Regressor:** 0.974097
- **KNN Regressor:** 1.0
- **R2 Test:** This metric measures how well the model generalizes to unseen data. Higher values indicate better generalization.
 - **Linear Regressor:** 0.667929
 - **Polynomial Regressor:** 0.664743
 - **Decision Tree Regressor:** 0.6
 - **Random Forest Regressor:** 0.6
 - **KNN Regressor:** 0.4

Observations:

- **Overfitting:** The Polynomial Regressor and KNN Regressor have perfect R2 scores on the training data, indicating overfitting. They may not generalize well to new data.
- **Better Generalization:** The Linear Regressor and Random Forest Regressor have similar R2 scores on both training and test data, suggesting better generalization.

Mean Squared Error (MSE):

- **MSE:** This metric measures the average squared difference between the predicted and actual values. Lower values indicate better predictions.
 - **Linear Regressor:** 6.18882
 - **Polynomial Regressor:** 2.00
 - **Decision Tree Regressor:** 2.02582
 - **Random Forest Regressor:** 2.00
 - **KNN Regressor:** 2.256426

Observations:

- **Lower MSE:** The Polynomial Regressor and Random Forest Regressor have the lowest MSE, indicating better predictive accuracy.

Mean Absolute Error (MAE):

- **MAE:** This metric measures the average absolute difference between the predicted and actual values. Lower values indicate better predictions.
 - **Linear Regressor:** 1.082650
 - **Polynomial Regressor:** 0.557663
 - **Decision Tree Regressor:** 1.25
 - **Random Forest Regressor:** 0.557663
 - **KNN Regressor:** 3.552256

Observations:

- **Lower MAE:** The Polynomial Regressor and Random Forest Regressor have the lowest MAE, indicating better predictive accuracy.
- **Based on R2 scores and MSE/MAE, the Polynomial Regressor and Random Forest Regressor seem to perform the best.**

Conclusion:

- After optimization with GridSearchCV, RandomizedSearchCV, and regularization techniques like Ridge, Lasso, and ElasticNet, both the **Linear Regressor** and **Polynomial Regressor** demonstrated strong performance. However, their susceptibility to overfitting, especially in complex models, should be considered.
- The **Random Forest Regressor**, optimized with GridSearchCV, RandomizedSearchCV, and Bayesian techniques like Optuna, proved to be a robust choice. Its ensemble nature and ability to handle nonlinear relationships contribute to its strong performance and generalization capabilities.
- To gain insights into the models' decision-making processes, techniques like SHAP, Partial Dependence Plots, and LIME were employed. These techniques help

identify important features, understand feature interactions, and explain predictions.

Future Directions:

- **Time Series Analysis:** If data has a temporal component, consider using time series forecasting techniques like ARIMA or Prophet.
- **Deep Learning:** Explore deep learning models, such as Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks, for complex time series or sequential data.